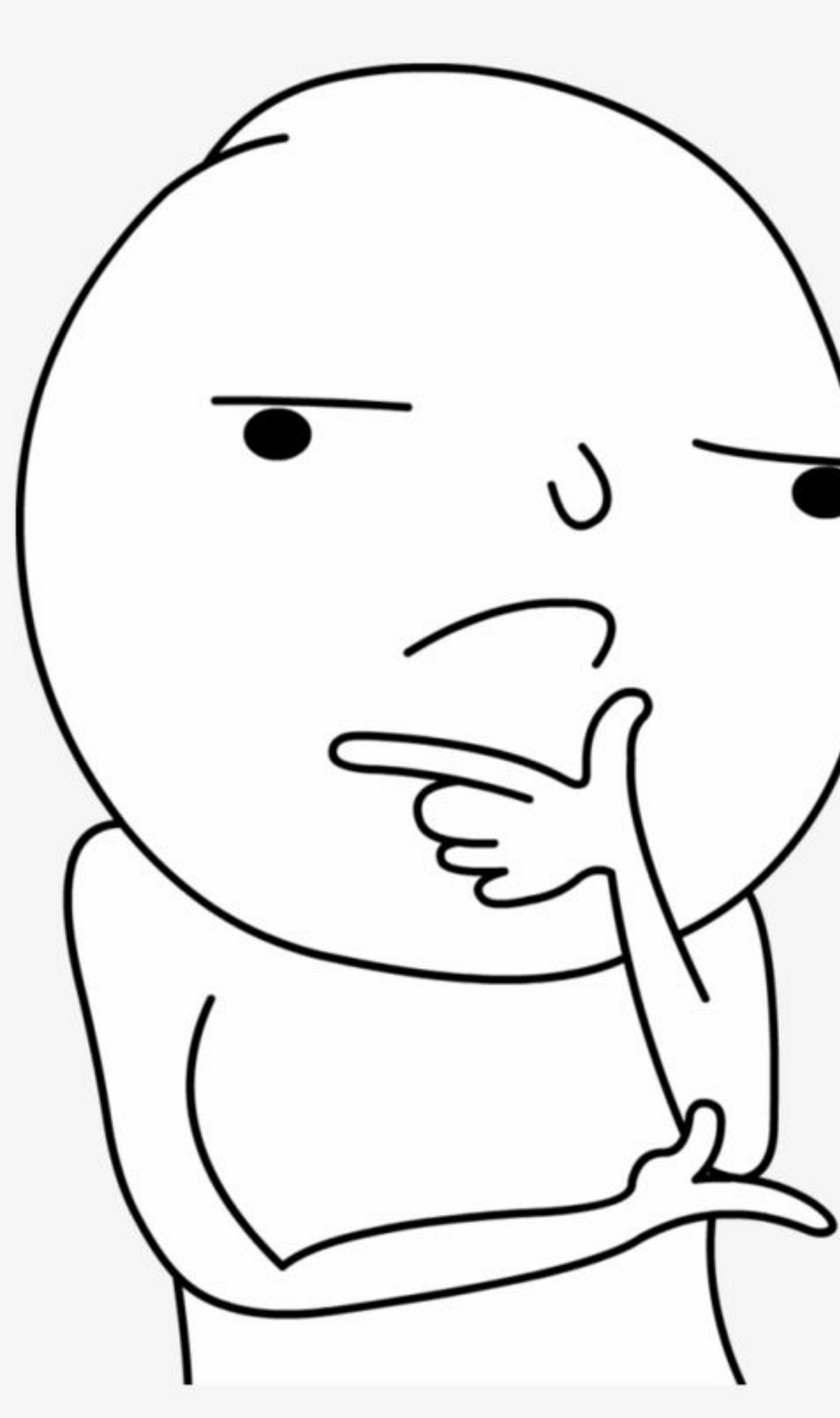
education

LLM Agents

Егор Спирин

1. Заставить модель рассуждать
2. Исправлять ошибки за собой
3. Использовать внешние инструменты
4. Взаимодействие с другими БЯМ



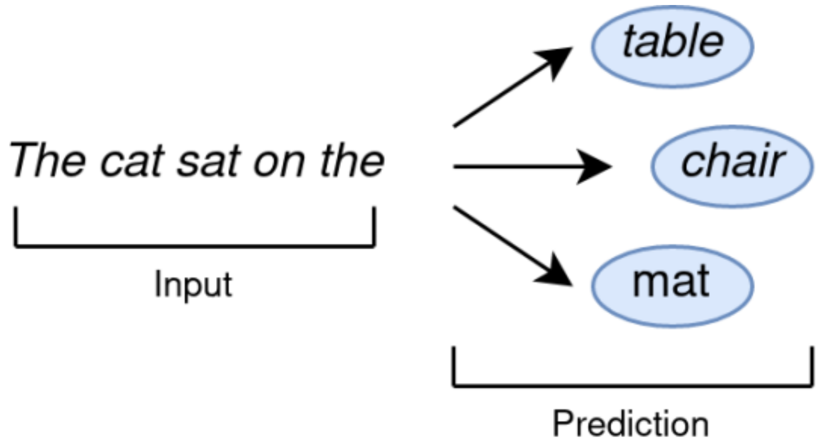
Quick Recap 📝

Языковые модели

Умеем предсказывать следующий токен 💅

Модель обучается в несколько стадий:

1. Pre-Train — много текста, “впитывание” знаний
2. Instruction Tuning — добавляем инструкции, “пользуемся” знаниями
3. Preference Optimization — выравниваем ответы, “нравимся” пользователю



GPT-2, 3, …

“Our largest model, GPT-2, is a 1.5B parameter Transformer that achieves state of the art results on 7 out of 8 tested language modeling datasets in a zero-shot setting but still underfits WebText”

Модель обученная на предсказывание следующего слова оказалась SOTA:

👉 POS-tagging
👉 Summarization
👉 Translation
👉 …

И все это без обучения под задачи против специально обученных моделей 🔥

[1] — Language Models are Unsupervised Multitask Learners, Alec Radford, Jeffrey Wu, Dario Amodei, Ilya Sutskever et al., OpenAI, 2019

Prompt Crafting 🏗️

Prompt a.k.a. Затравка

Текстовый запрос или инструкция, подаваемая языковой модели для генерации ответа или выполнения задачи

- 👉 Управление моделью
- 👉 Передача знаний
- 👉 Позволяют раскрыть весь потенциал БЯМ

Размер контекста ограничен моделью ⇒ размер промпта также ограничен

```
Question: how many apples in the bucket?

Answer: there are 52 apples.
```

Few-shot

Можем показать модели несколько примеров с ожидаемым поведением

Примеры с паттерном

Сама задача

```
Question: Elon Musk
Answer: nk

Question: Bill Gates
Answer: ls

Question: Sam Altman
Answer: ??
```

Человек легко ответить — он умеет размышлять (reasoning), в отличие от LLM 🫠

Chain-of-Thought

Давайте добавим шаг с рассуждением!

Question: Elon Musk
Answer: the last letter of "Elon" is "n". The last letter of "Musk" is "k". Concatenating "n", “k" leads to "nk". So the output is "nk".

Question: Bill Gates
Answer: the last letter of "Bill" is "l". The last letter of "Gates" is "s". Concatenating "l", "s" leads to "ls". So the output is "ls".

Question: Sam Altman
Answer: the last letter of "Sam" is “m". The last letter of "Altman" is "n". Concatenating "m", "n" leads to "mn". So the output is "mn".

Zero-Shot CoT

Формируй промпты правильно 👍

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A:

(Output) *The juggler can juggle 16 balls. Half of the balls are golf balls. So there are 16 / 2 = 8 golf balls. Half of the golf balls are blue. So there are 8 / 2 = 4 blue golf balls.* *The answer is 4.* ✓

(d) Zero-shot-CoT (Ours)

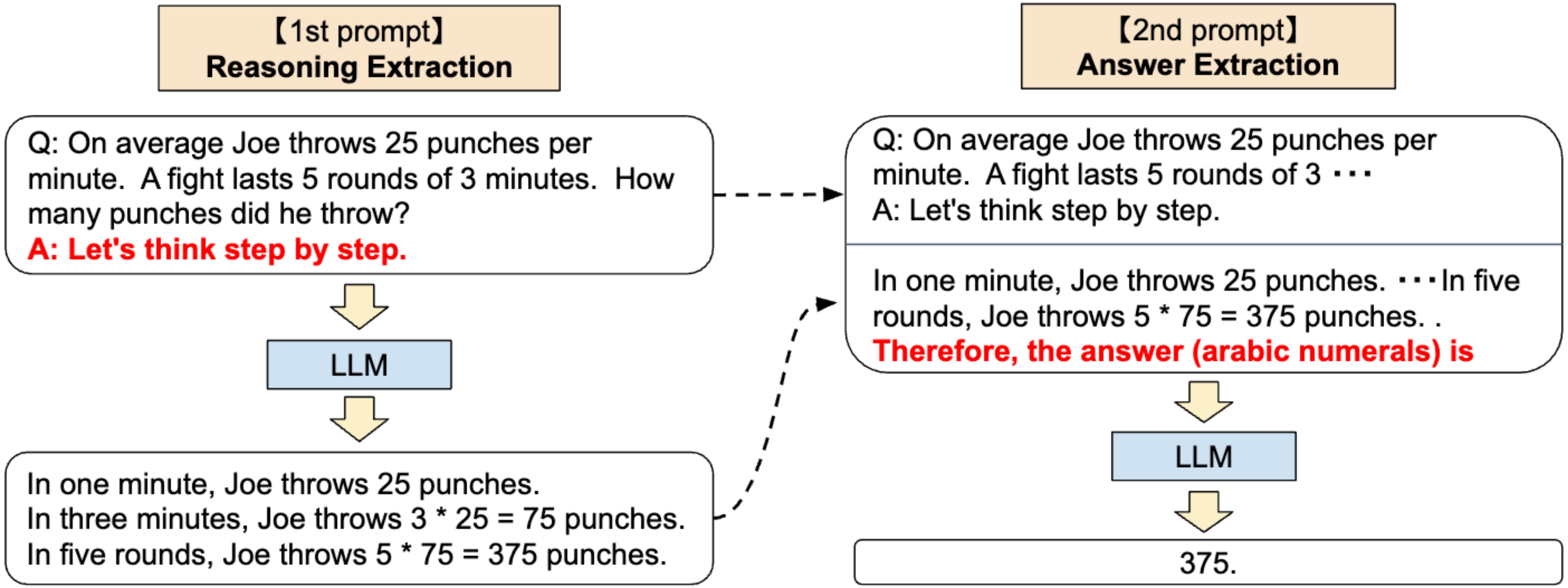
Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?
A: ***Let's think step by step.***

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls.* ✓

[3] — Large Language Models are Zero-Shot Reasoners, Kojima et al., Google Research, 2023

Reason → Answer

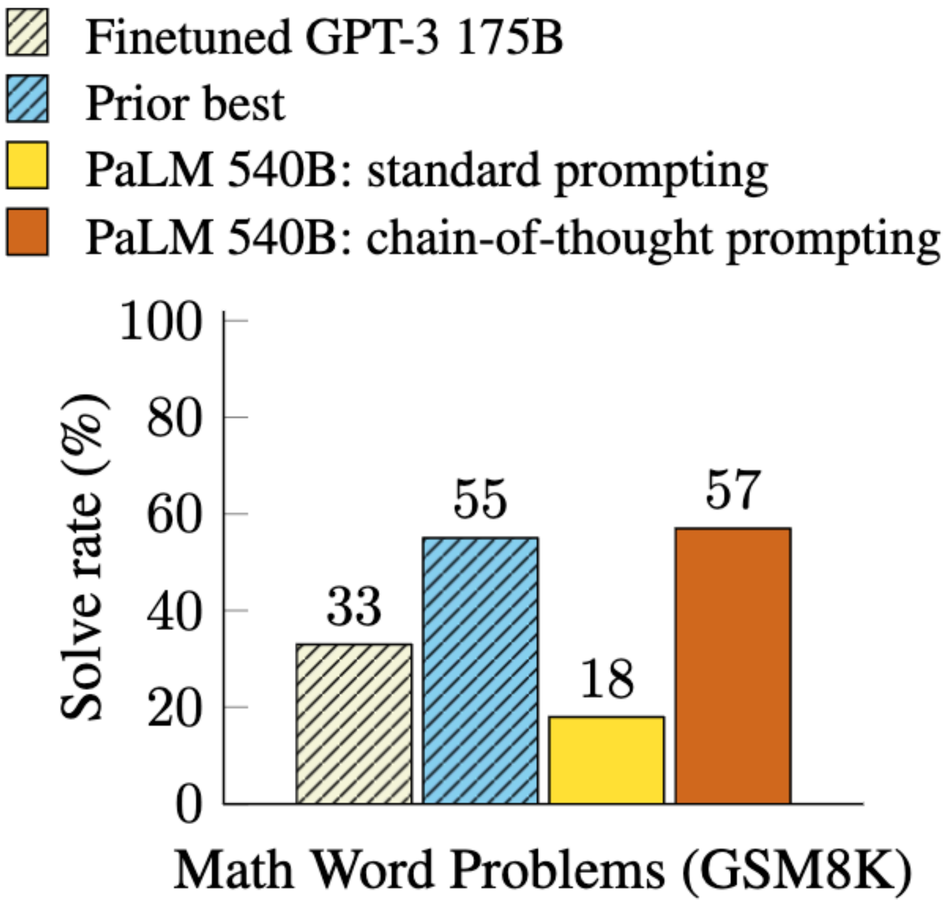
1. Парсить вывод через ключевые слова
2. Использовать структурную генерацию
3. Переиспользовать LLM



Применимость

Особенно хорошо справляется с математическими задачами!

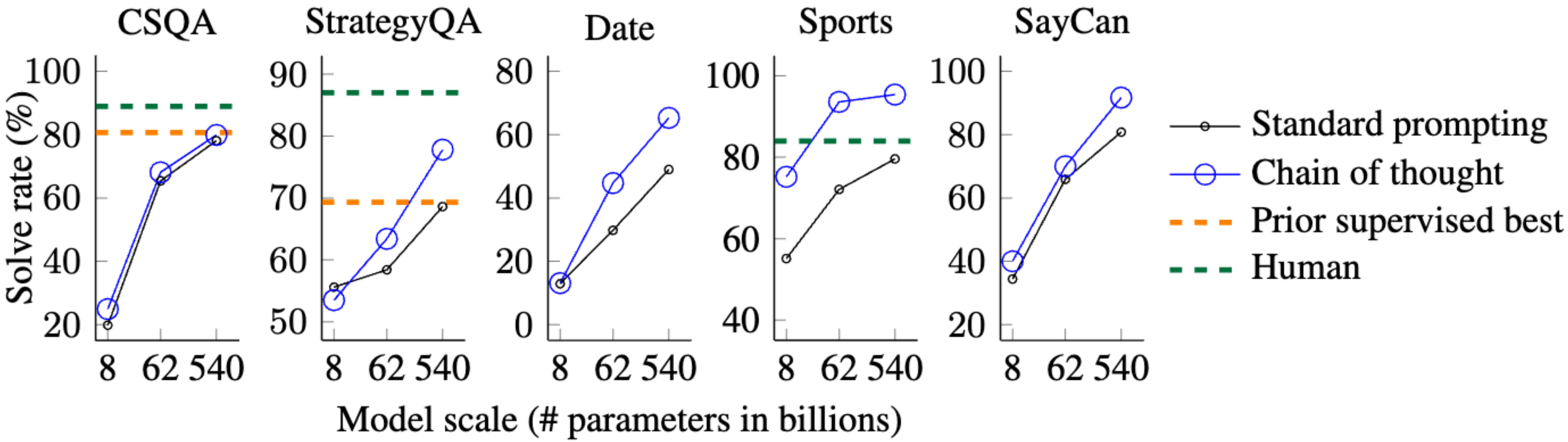
- 👉 Дообучение GPT-3 под задачу
- 👉 Лучшая модель на бенчмарке на 2022 год
- 👉 Стандартный промптинг — few-shot
- 👉 Chain-of-Thought



[2] — Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, Wei et al., Google Research, 2023

Применимость

На самом деле лучше везде, если необходим “reasoning”



[2] — Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, Wei et al., Google Research, 2023

Chain-of-Though | Выводы

Плюсы:

👉 Улучшаем качество на reasoning задачах

👉 Не требует дообучения

👉 Получаем более интерпретируемый результат

Chain-of-Though | Выводы

Плюсы:

- 👉 Улучшаем качество на reasoning задачах
- 👉 Не требует дообучения
- 👉 Получаем более интерпретируемый результат

Минусы:

- 👉 Увеличиваем число токенов на генерации и в промпте ⇒ дольше работает
- 👉 Это не панацея, если LLM плохая, то не поможет

Интуиция и лайфхаки

- 👉 LLM лучше понимает, если использовать знакомый язык и конструкции
- 👉 LLM очень легко отвлекается
  Не надо добавлять много информации “на всякий случай”
- 👉 Если информации не было на обучение или в промпте, то LLM не ответит **верно**
- 👉 Если вы смотрите на промпт и не понимаете, что требуется, то и LLM тоже :)
- 👉 LLM не умеет сама исправлять размышления
  Если ушла не туда, то там и останется

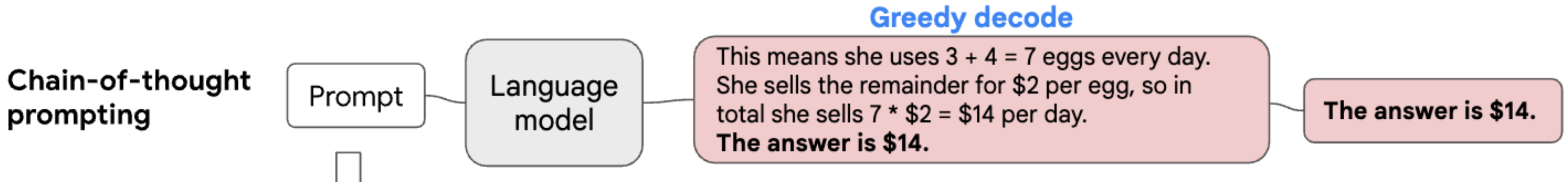
Работа над ошибками 📏

Self-Consistency Decoding

LLM: строит распределение по словарю

USER: берет жадно или семплирует

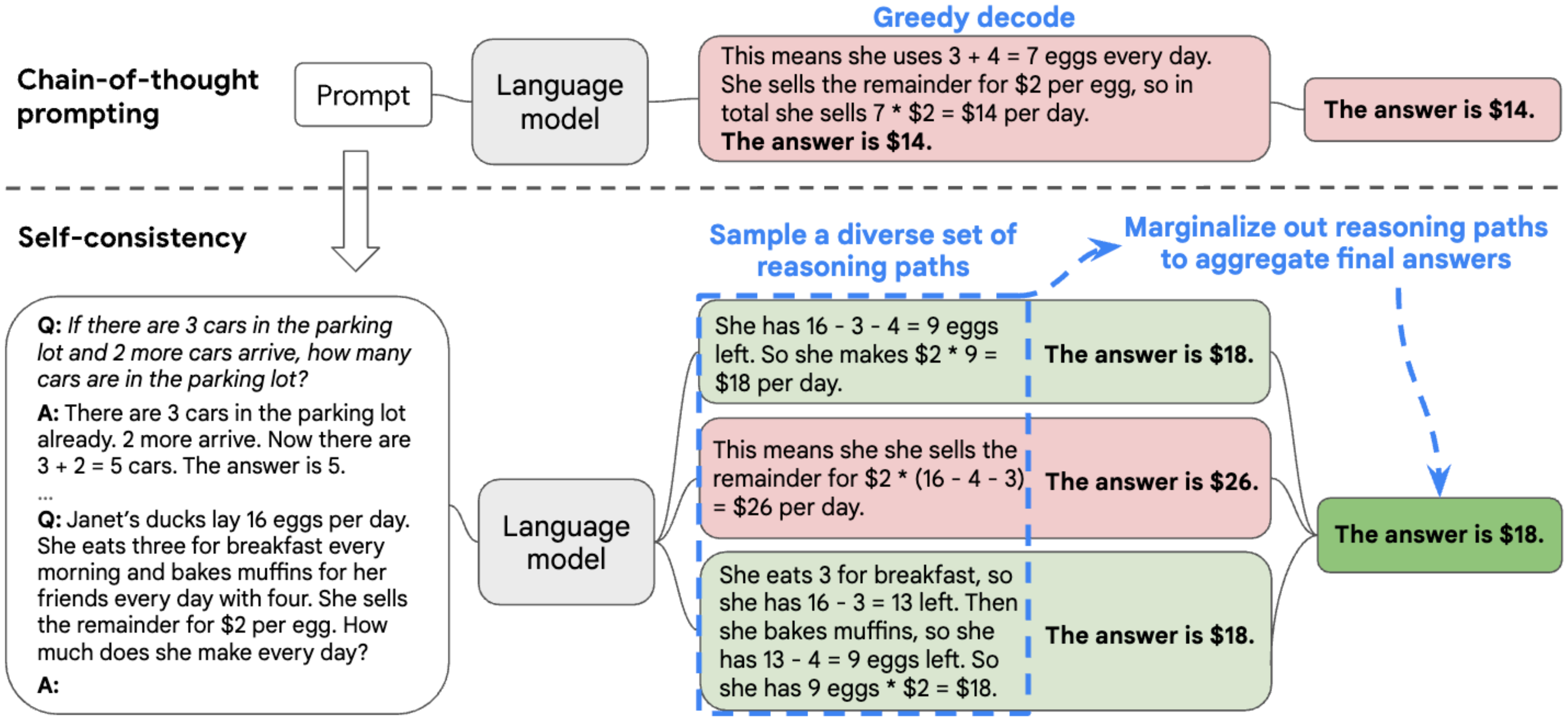
А где гарантия???



Self-Consistency Decoding

Не ошибается лишь тот, кто ничего не делает 🐺

Давайте дадим второй шанс, или третий…



[4] — Self-Consistency Improves Chain of Thought Reasoning in Language Models, Wang et al., Google Research, 2023

Free-form Generation

Question: Where do people drink less coffee than they do in Mexico?
Answer: ... Some examples include Japan, China and the United Kingdom...

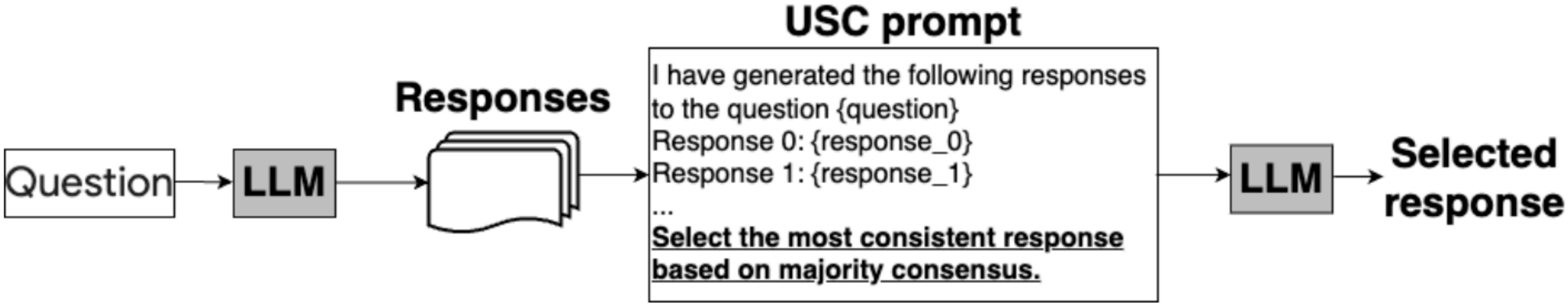
Question: Where do people drink less coffee than they do in Mexico?
Answer: ... People in countries like Japan, China, and India typically drink less coffee...

Question: Where do people drink less coffee than they do in Mexico?
Answer: ... There are several countries where people generally drink less compared to Mexico. Some of these countries include:
1. Japan: ...
2. China: ...
3. Saudi Arabia: ...
4. India: ...
...

Не всегда задачу можно формализовать для понятного извлечения ответа после размышления 🫠

Universal Self-Consistency

Попросим LLM саму выбрать ответ 🙃



[5] — Universal Self-Consistency for Large Language Model Generation, Xinyun Chen, Renat Aksitov et al., DeepMind, 2023

Analogical Prompting

Zero-Shot CoT все равно хуже, чем Few-Shot CoT 😥

💡 Давайте сначала сгенерируем похожие задачи

👉 Теперь примеры самогенерируются, не нужна ручная разметка

👉 Примеры лучше подходят под задачу

Model Input

Q: What is the area of the square with the four vertices at (-2, 2), (2, -2), (-2, -6), and (-6, -2)?

Instruction:
Recall relevant exemplars:
Solve the initial problem:

Model Output

 # Relevant exemplars:
Q: What is the area of the square with a side length of 5?
A: The area of a square is found by squaring the length of its side. So, the area of this square is 5^2 = 25. ...

 # Solve the initial problem:
To find the area of the square, we need to find the side length. The length is … \sqrt{(2 - (-2))^2 + (-2 - 2)^2} = \sqrt{32}. So, the area of the square is (\sqrt{32})^2 = 32.

[5] — Large Language Models as Analogical Reasoners, Michihiro Yasunaga et al., ICLR'24

Analogical Prompting | Codeforces

Our Prompt: Self-generate Exemplars + Knowledge

```
Your goal is to write Python3 code to solve competitive programming problems. Given a problem, explain the core concepts in it and provide
other relevant problems. Then solve the original problem.

# Problem:
You are given a sequence a_1, a_2, …, a_n. Each element of a is 1 or 2. Find out if an integer k exists so that the following conditions are
met. 1 ≤ k ≤ n-1, and a_1 · a_2 … a_k = a_{k+1} … a_n. If there exist multiple k that satisfy the given condition, print the smallest.

# Instruction:
## Algorithms:
Identify the core concepts or algorithms used to solve the problem.

## Tutorial:
Write a tutorial about these algorithms.

## Example Problems:
Provide three examples of relevant competitive programming problems that involve these algorithms. For each problem, describe the problem,
explain the solution in detail, and then write the correct Python3 code.

## Python3 code to solve the original problem:
- Explanation of the solution:
- Python3 code to solve the problem:
```

Помимо генерации примера еще просим добавить необходимые знания — алгоритм и его описание

Analogical Prompting | Codeforces

Output by GPT3.5-turbo

Self-generated Knowledge

Algorithms: The core algorithm for solving this problem is the prefix product algorithm.

Tutorial: The prefix product algorithm is used to calculate the product of all the elements in a prefix of an array. The algorithm works by initializing a variable "prefix" to 1, and then iterating through the array from left to right. At each iteration, the current element is multiplied with the prefix, and the result is stored in the prefix variable. This way, the prefix variable keeps track of the product of all the elements in the prefix of the array. The time complexity of this algorithm is O(n), where n is the size of the input array. ......

Self-generated Exemplars

Example Problems:

1. Problem: Given an array of integers, find the product of all the elements in each subarray of size k.
Solution: We can use the prefix product algorithm to solve this problem. We initialize a prefix variable to 1, and then iterate through the array from left to right. At each iteration, we multiply the current element with the prefix, ...

```
def product_of_subarrays(arr, k):
    prefix = 1
    result = []
    for i in range(len(arr)):
        prefix = prefix * arr[i]
        if i >= k-1:
            result.append(prefix)
            prefix = prefix // arr[i-k+1]
    return result
```

2. Problem: Given an array of integers, find the longest subarray with a product less than or equal to a given target.
Solution: ......

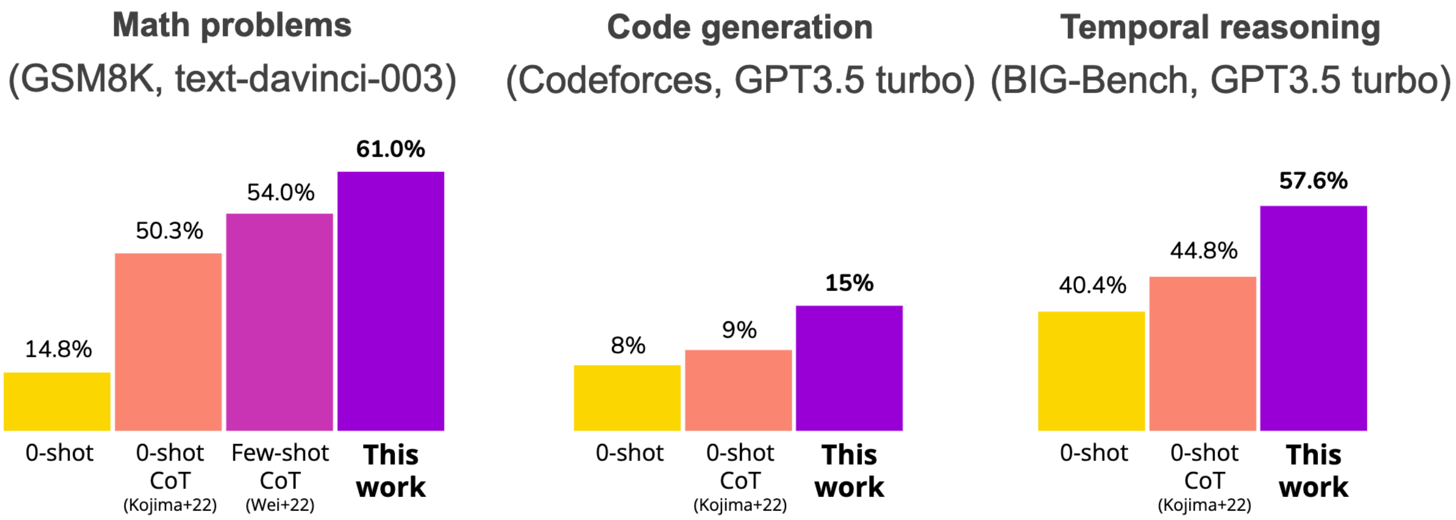
3. Problem: Given an array of integers, find the smallest k such that the product of all the elements in the prefix of the array is equal to the product of all the elements in the suffix of the array.
Solution: ......

Analogical Prompting | Codeforces

```
## Python3 code to solve the original problem:
Explanation of the solution:
  - Initialize prefix and suffix variables to 1, and k to -1.
  - Calculate the product of all the elements in the sequence using the prefix product algorithm.
  - Iterate through the sequence from right to left.
    - Update the prefix and suffix variables.
    - If the prefix is equal to the suffix, update k to the current index.
  - Print the value of k.

t = int(input())
for _ in range(t):
    n = int(input())
    arr = list(map(int, input().split()))
    prefix, suffix = 1, 1
    k = -1
    for i in range(n):
        prefix = prefix * arr[i]
    for i in range(n-1, 0, -1):
        prefix = prefix // arr[i]
        suffix = suffix * arr[i]
        if prefix == suffix:
            k = i
    print(k)
```

Analogical Prompting | Выводы

- 👉 Работает лучше, чем примеры подобранные вручную
- 👉 Хорошо скейлится с моделями, отлично работает именно с большими умными моделями



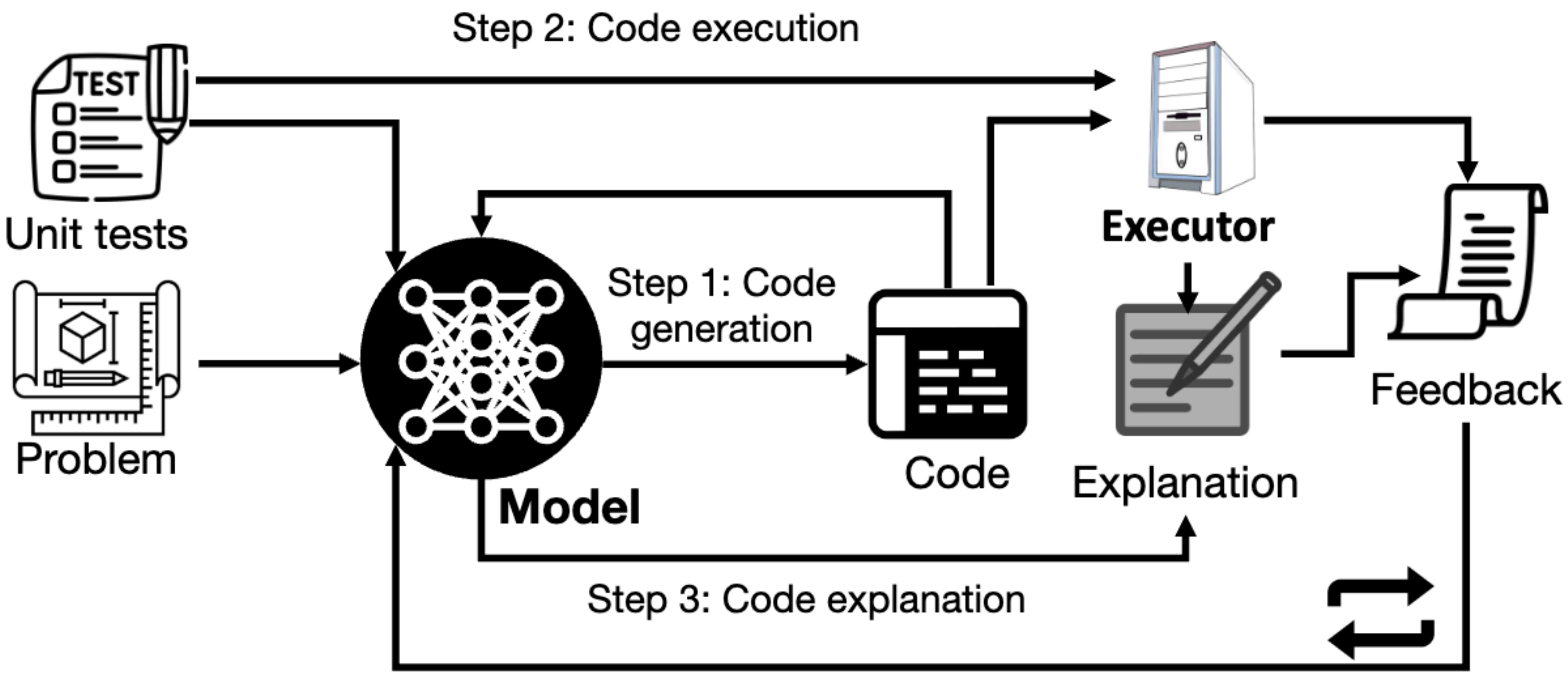
Prompting Method	(← scale down) text-curie-001	text-davinci-001	text-davinci-002	**(scale up →) text-davinci-003**
0-shot	2%	6%	13%	14%
0-shot CoT	2%	6%	22%	50%
5-shot (fixed) CoT	2%	10%	43%	54%
5-shot retrieved CoT	3%	11%	47%	57%
Ours: Self-generated Exemplars	2%	9%	**48%**	**61%**

GSM8K for math reasoning

Oracle feedback

Одно из существенных ограничений — LLM не понимает, если ошибается в выводах 😥

1. Модель генерирует код
2. Код исполняется и проверяется тестами
3. Модель объясняет, что код делает
4. Формируется фидбек и начинается следующая итерация дебага



[6] — Teaching Large Language Models to Self-Debug, Xinyun Chen et al., DeepMind, 2023

Self-debugging Improves Code Quality

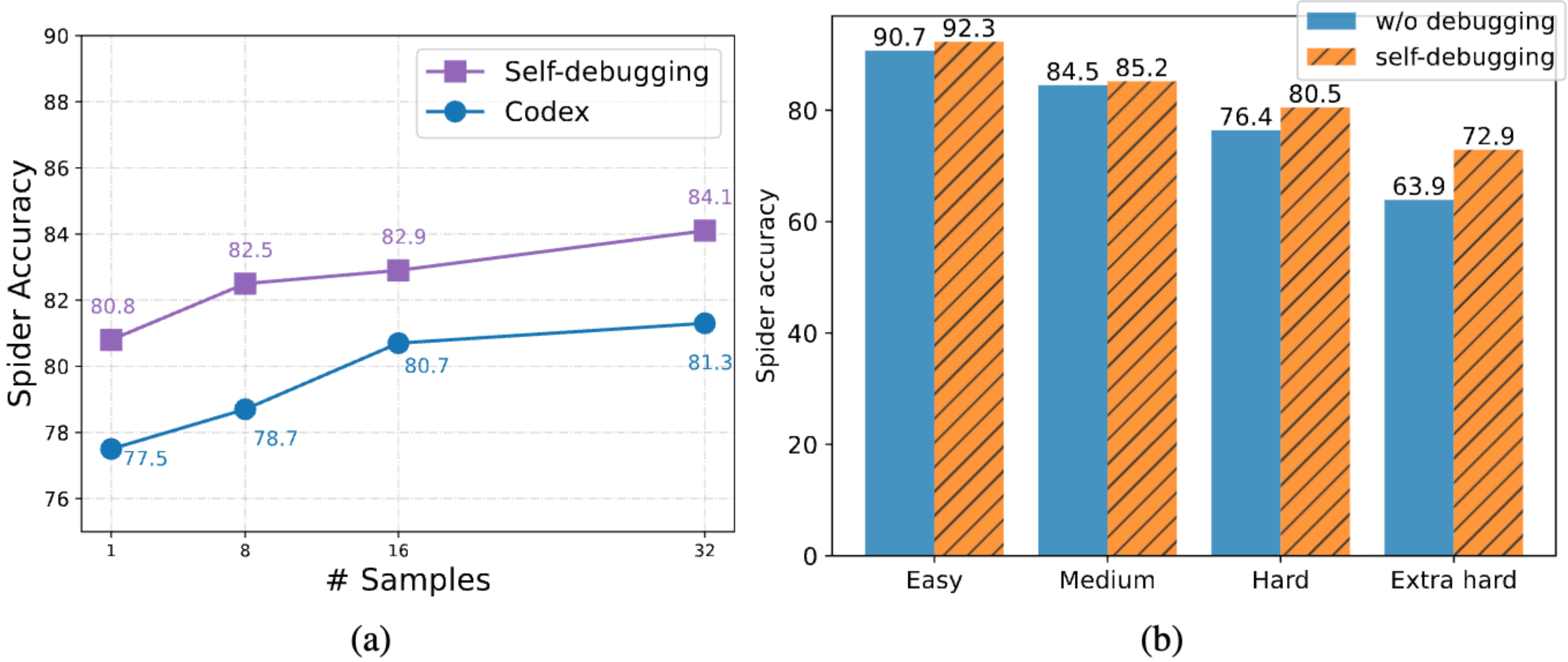


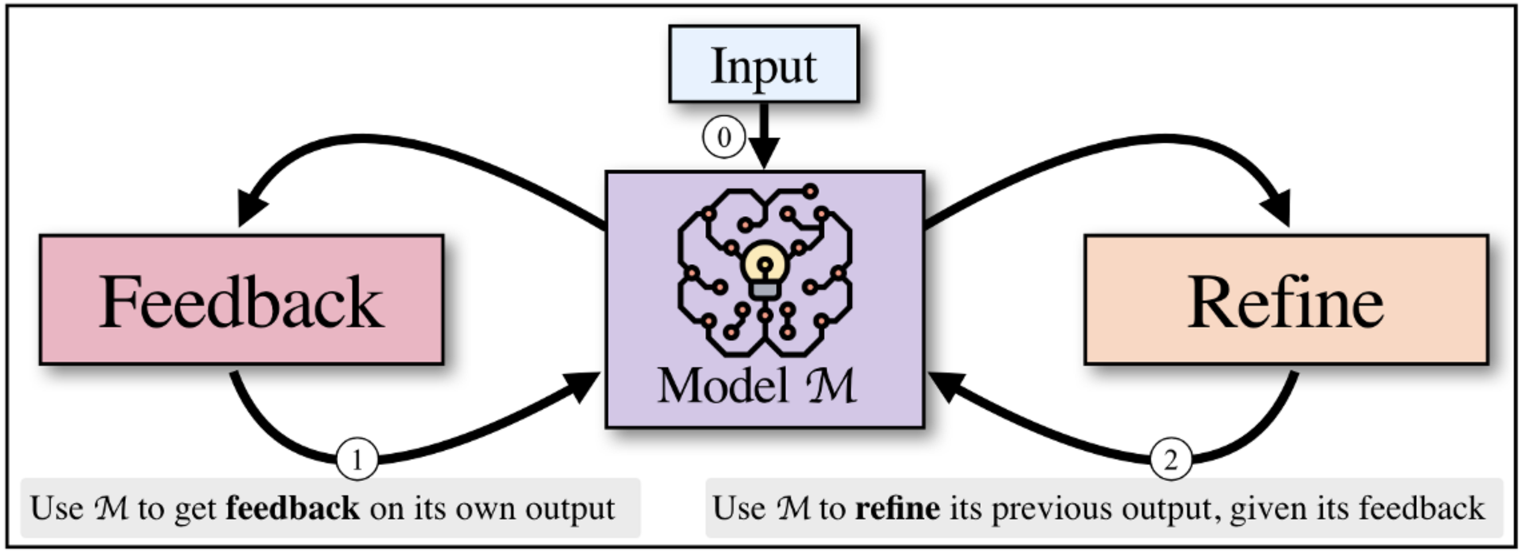
Figure 6: Ablation studies on the Spider development set with Codex. (a) Accuracies with different numbers of initial samples. (b) Breakdown accuracies on problems with different hardness levels.

[6] — Teaching Large Language Models to Self-Debug, Xinyun Chen et al., DeepMind, 2023

Self-Refine

Мы снова делаем это — просим LLM исправить свои же ошибки…

Input (0) передается в модель, ответ (1) передается снова в модель для получения фидбека (2). Все повторяется пока не встретится критерий останова.



[7] — Self-Refine: Iterative Refinement with Self-Feedback, Aman Madaan et al., 2023

Пример FeedBack & Refine | Диалог

(a) **Dialogue:** x, $\mathbf{y_t}$

User: I am interested in playing Table tennis.

Response: I'm sure it's a great way to socialize, stay active

(b) **FEEDBACK** $\quad$ **fb**

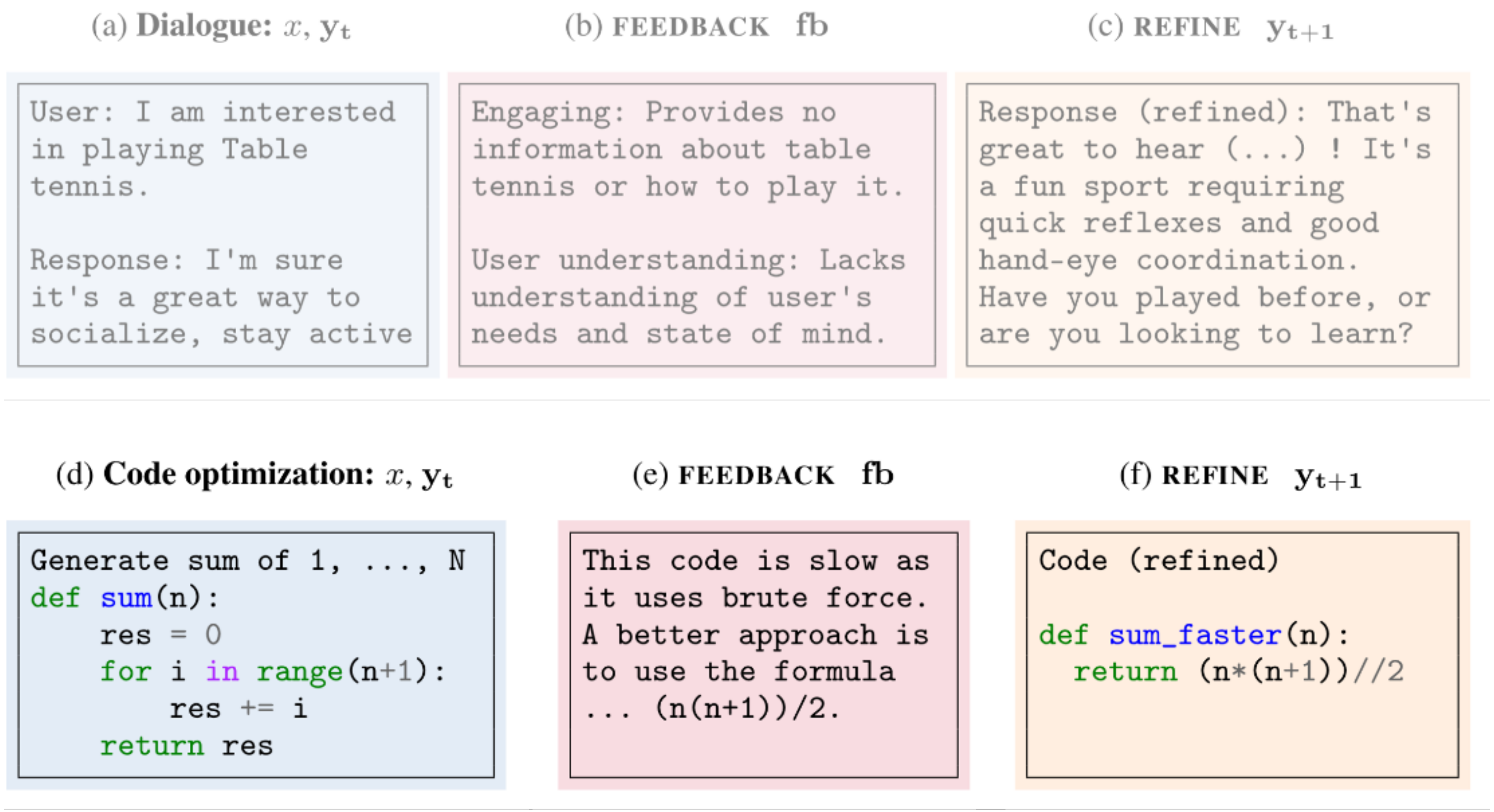
Engaging: Provides no information about table tennis or how to play it.

User understanding: Lacks understanding of user's needs and state of mind.

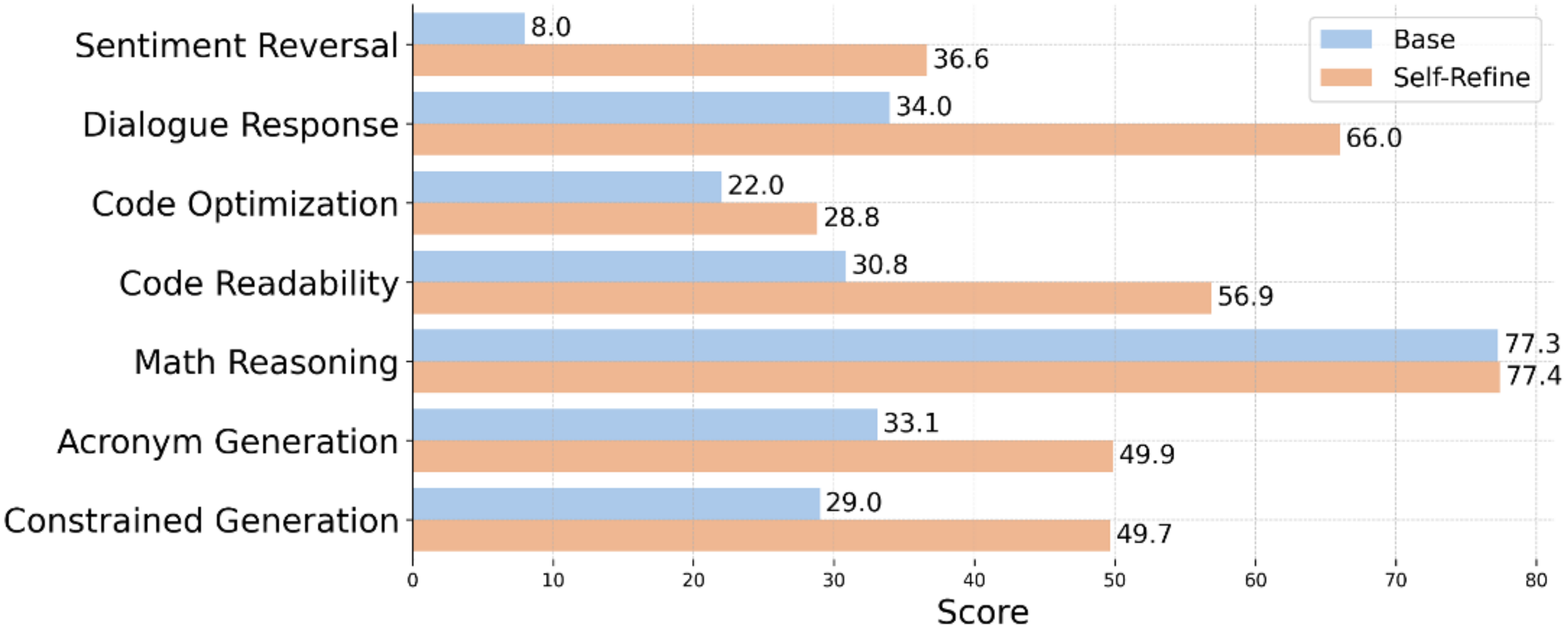
(c) **REFINE** $\quad$ $\mathbf{y_{t+1}}$

Response (refined): That's great to hear (...) ! It's a fun sport requiring quick reflexes and good hand-eye coordination. Have you played before, or are you looking to learn?

Пример FeedBack & Refine | Генерация кода



Улучшаем качество на генеративных задачах



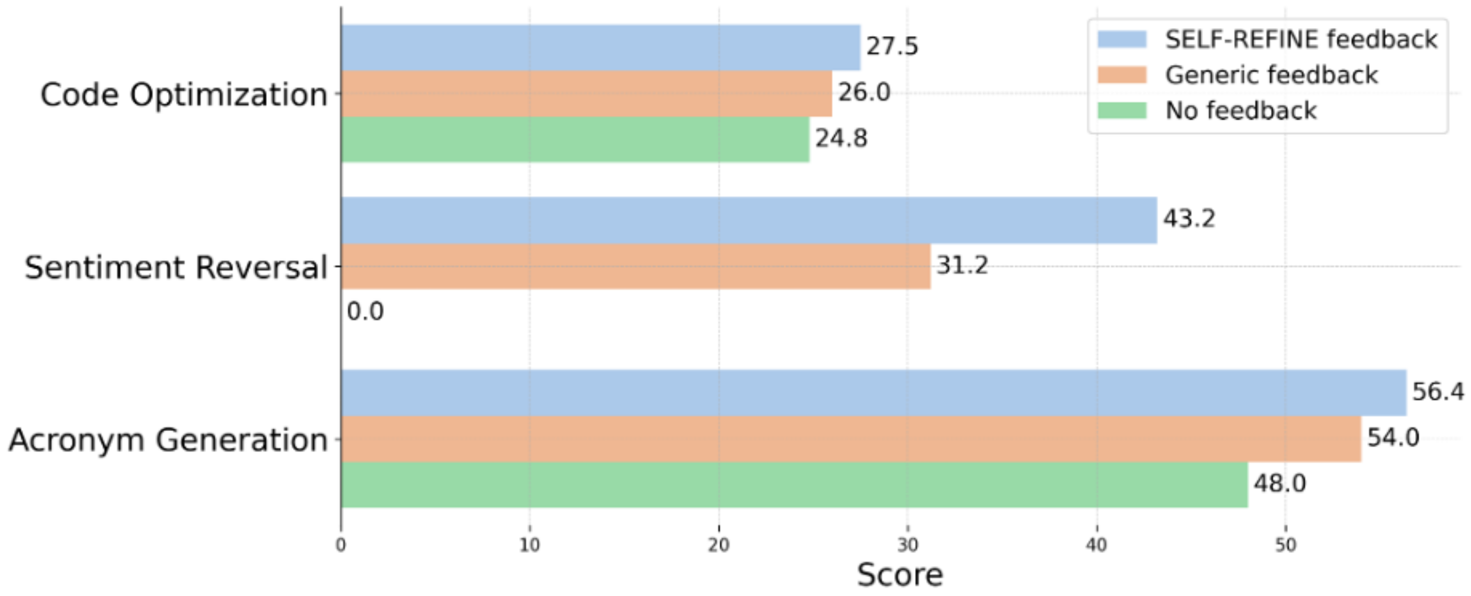
- 👉 Модели способны самостоятельно улучшать свои ответы
- 👉 Можно достичь более высокого качества
- 👉 На Math-Reasoning нет прироста — другая задача!

Важность feedback

Feedback должен быть

👉 Specific — выделять конкретные места ответа для исправления

👉 Actionable — предлагать конкретные действия для улучшения

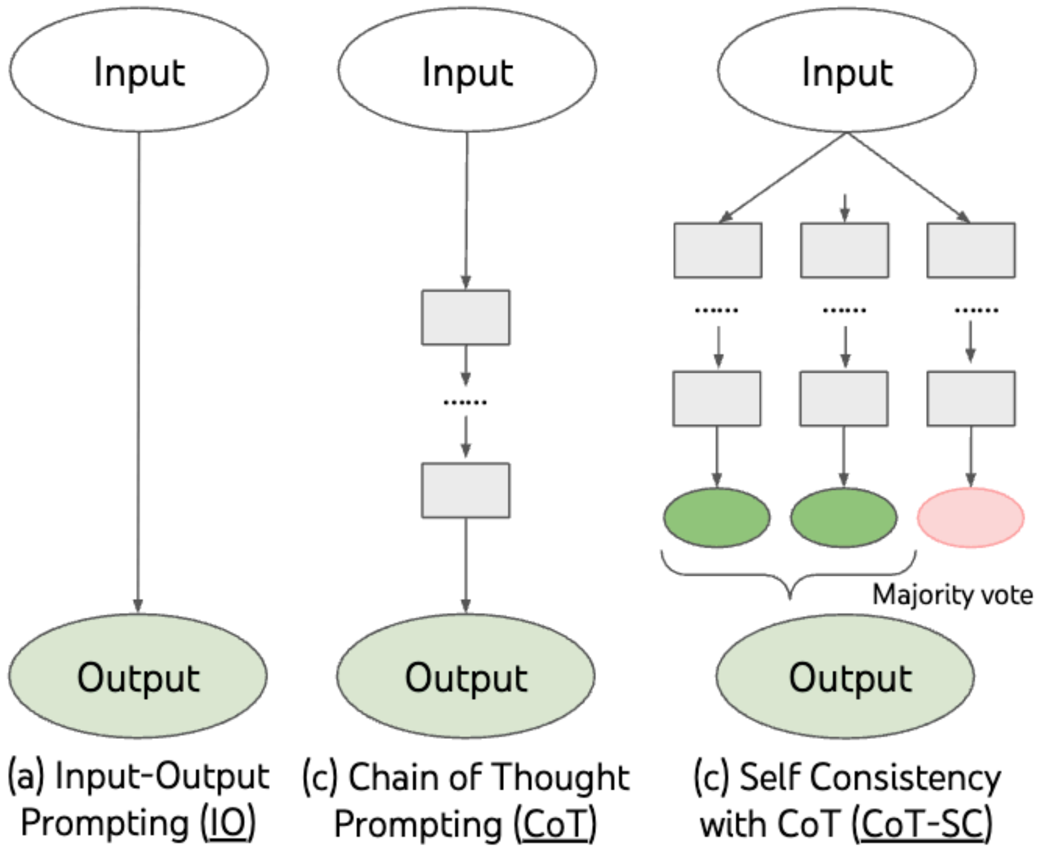


А что если?..

Умеем повышать качество reasoning и свободной генерации, но по отдельности...

Tree of Thoughts

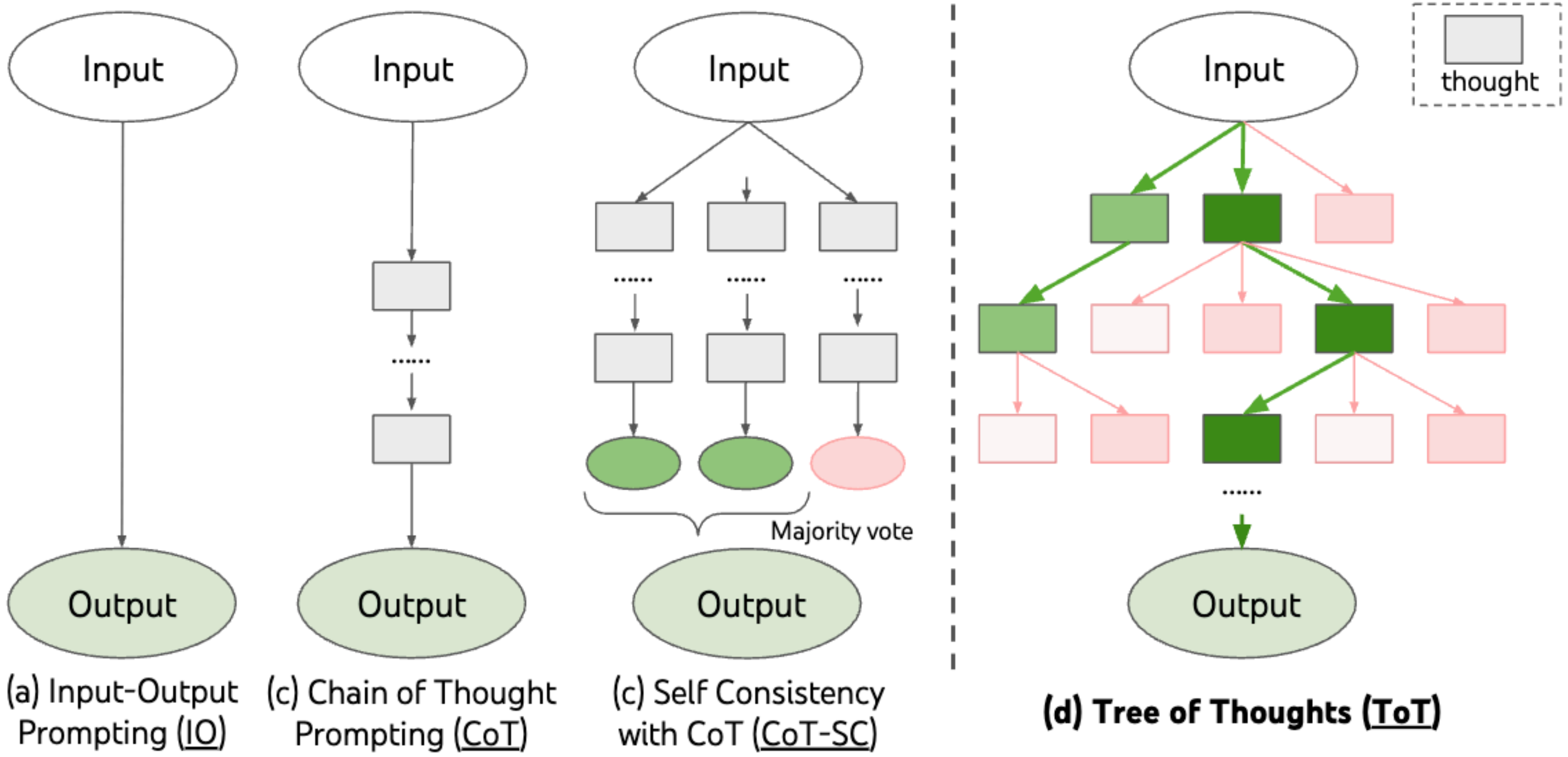
Это уже знаем



[8] — Tree of Thoughts: Deliberate Problem Solving with Large Language Models, Yao et al., Princeton & DeepMind, 2023

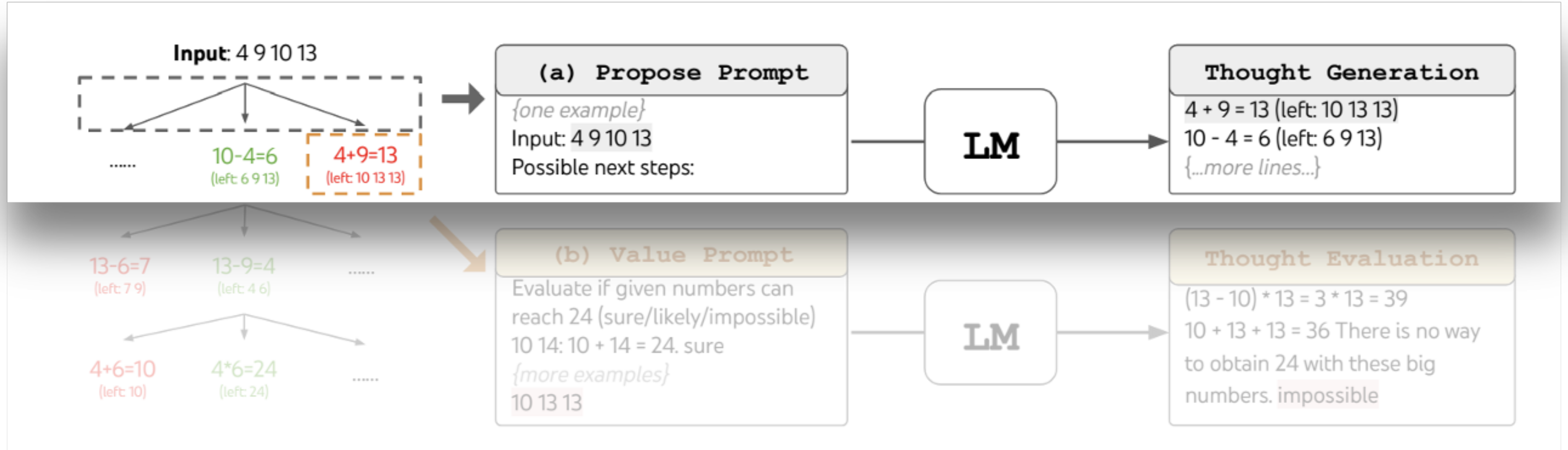
Tree of Thoughts

Это уже знаем



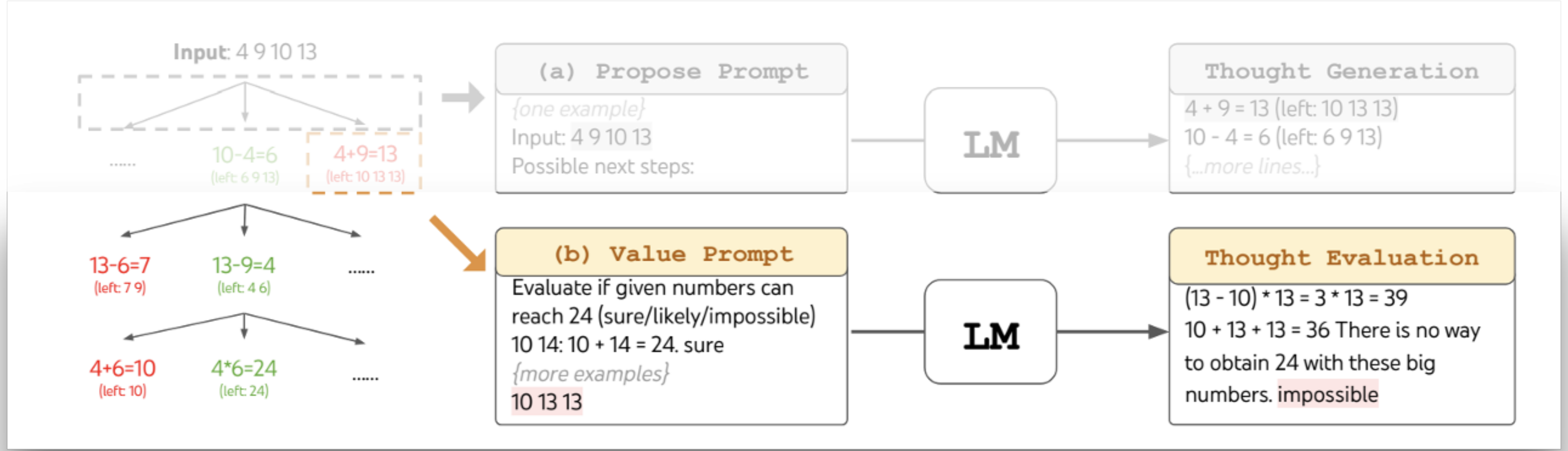
[8] — Tree of Thoughts: Deliberate Problem Solving with Large Language Models, Yao et al., Princeton & DeepMind, 2023

Game of 24



[8] — Tree of Thoughts: Deliberate Problem Solving with Large Language Models, Yao et al., Princeton & DeepMind, 2023

Game of 24



[8] — Tree of Thoughts: Deliberate Problem Solving with Large Language Models, Yao et al., Princeton & DeepMind, 2023

ReAct: Synergizing Reasoning and Acting

Агенты существуют в некотором окружении $\Rightarrow$ есть набор действий $\mathscr{A}$

- 👉 Классическая политика: $\pi(a_t \mid c_t)$, где $c_t = (o_1, a_1, \ldots, o_{t-1}, a_{t-1}, o_t)$
- 👉 Расширим пространство действий языковыми конструкциями $\hat{\mathscr{A}} = \mathscr{A} \cap \mathscr{L}$
- 👉 $\hat{a}_t \in \mathscr{L}$ – thought/reasoning, не влияет на внешнюю среду и поэтому не получает observation, но полезно для анализа контекста
- 👉 $\mathscr{L}$ – бесконечно и обучение в нем сложно, нужен сильный агент, например, LLM!

[9] — ReAct: Synergizing Reasoning and Acting in Language Models, Yao et al., Princeton & Google Research, ICLR'23

ReAct in a nutshell 🌰

1. Взяли PaLM-540B и запромптили few-shot примерами на генерацию и domain-specific действий, и free-form рассуждений
2. Каждый пример — последовательность действий, “мыслей” и наблюдений из окружения
3. Если задача явно требует рассуждений, то рассуждение чередуется с действием со средой
4. Если задача больше принятие решения, то модель сама выбирает какое действие надо

```
ReAct
Question         What is the elevation range for the area that the eastern sector of the
                 Colorado orogeny extends into?
Thought 1        I need to search Colorado orogeny, find the area that the eastern sector
                 of the Colorado orogeny extends into, then find the elevation range of the
                 area.
Action 1         Search[Colorado orogeny]
Observation 1    The Colorado orogeny was an episode of mountain building (an orogeny) in
                 Colorado and surrounding areas.
Thought 2        It does not mention the eastern sector.  So I need to look up eastern
                 sector.
Action 2         Lookup[eastern sector]
Observation 2    (Result 1 / 1) The eastern sector extends into the High Plains and is called
                 the Central Plains orogeny.
Thought 3        The eastern sector of Colorado orogeny extends into the High Plains.  So I
                 need to search High Plains and find its elevation range.
Action 3         Search[High Plains]
Observation 3    High Plains refers to one of two distinct land regions
Thought 4        I need to instead search High Plains (United States).
Action 4         Search[High Plains (United States)]
```

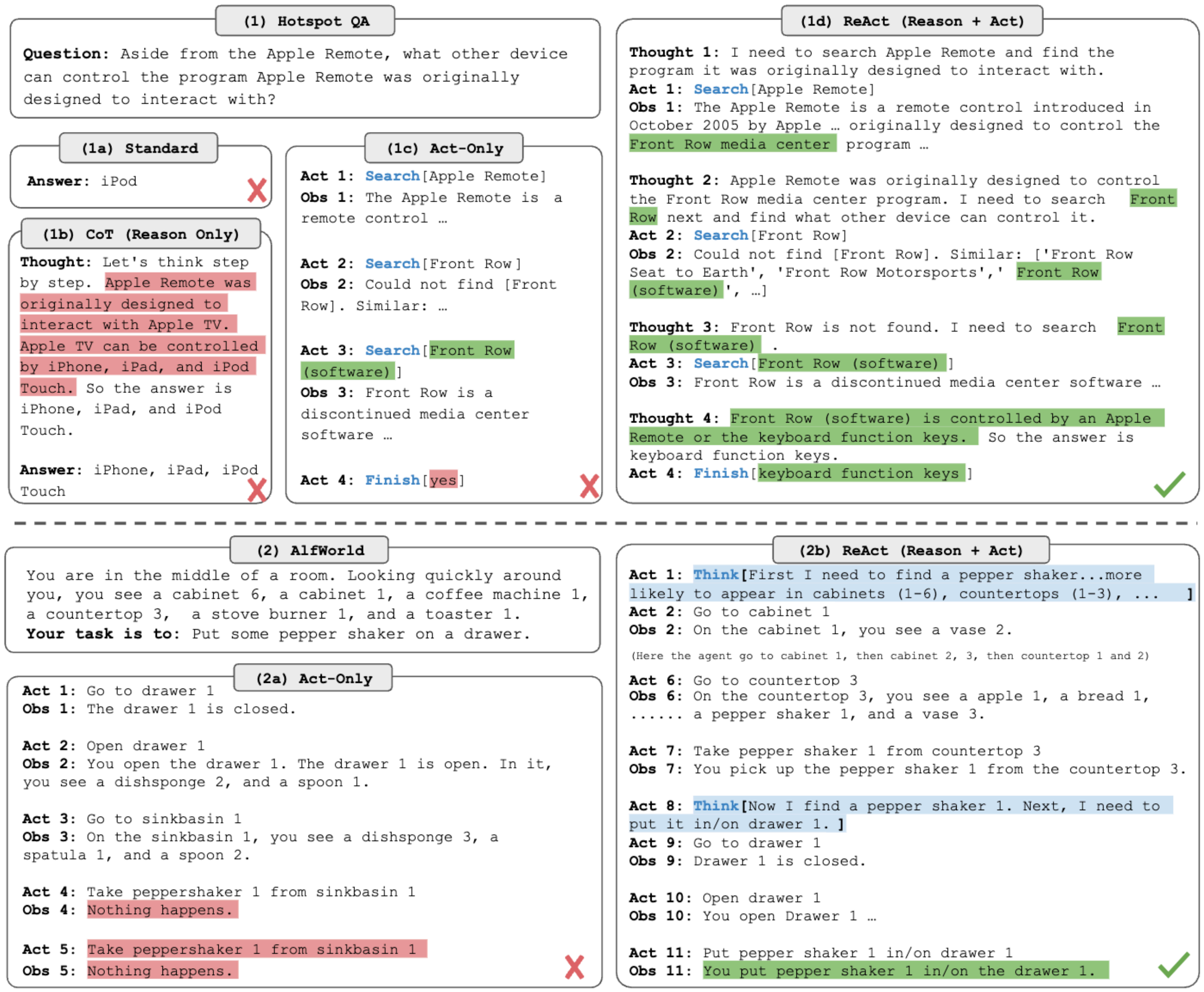
Continued on next page

ReAct — пример

HotPotQA — reasoning over two or more Wikipedia passages

Action Space: search, lookup, finish

AlfWorld — synthetic text-based game to interact with household



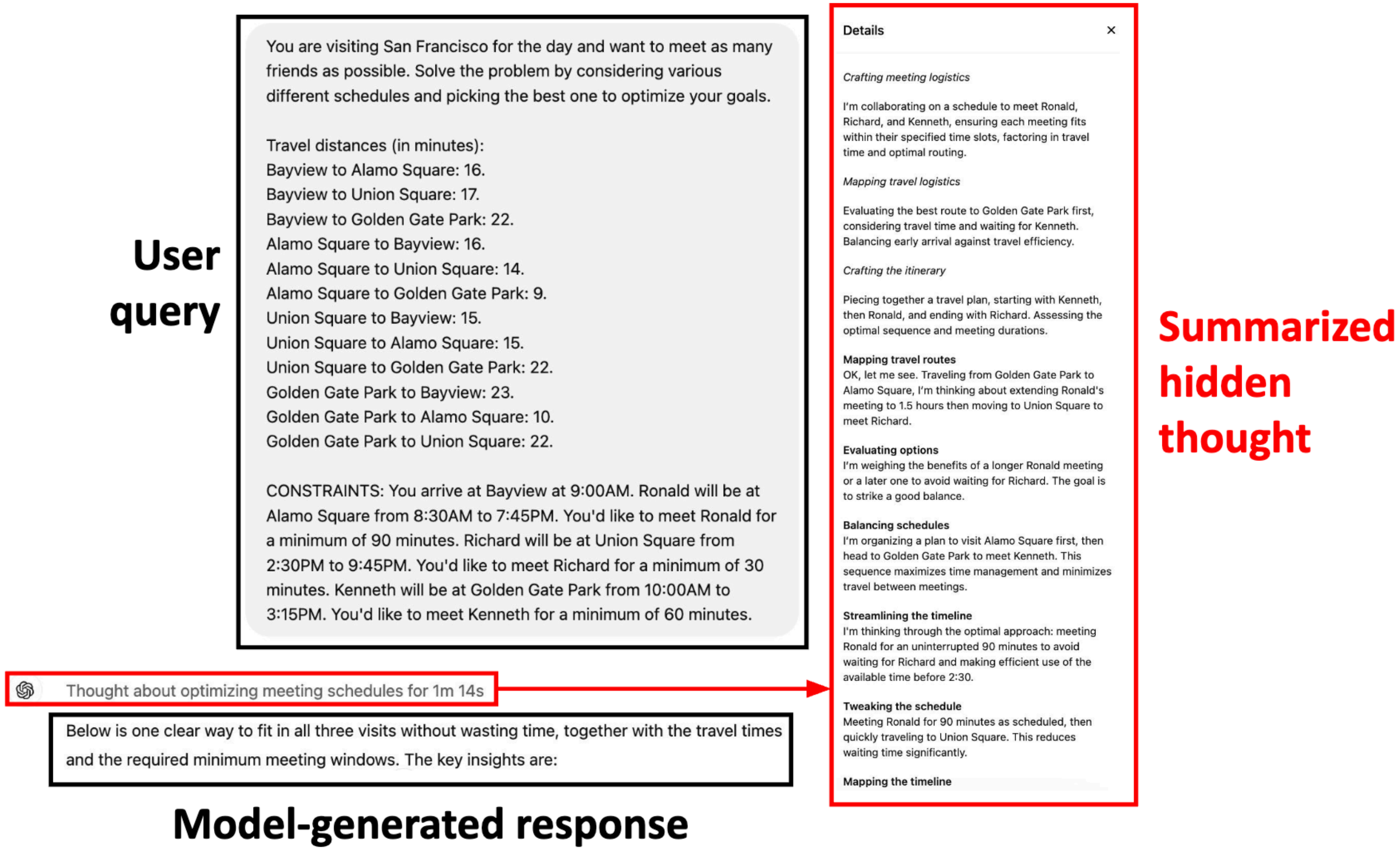
Inference-Time Scaling

Научились тратить время инференса на повышение качества LLM — нет обучения, есть стратегии извлечения знаний

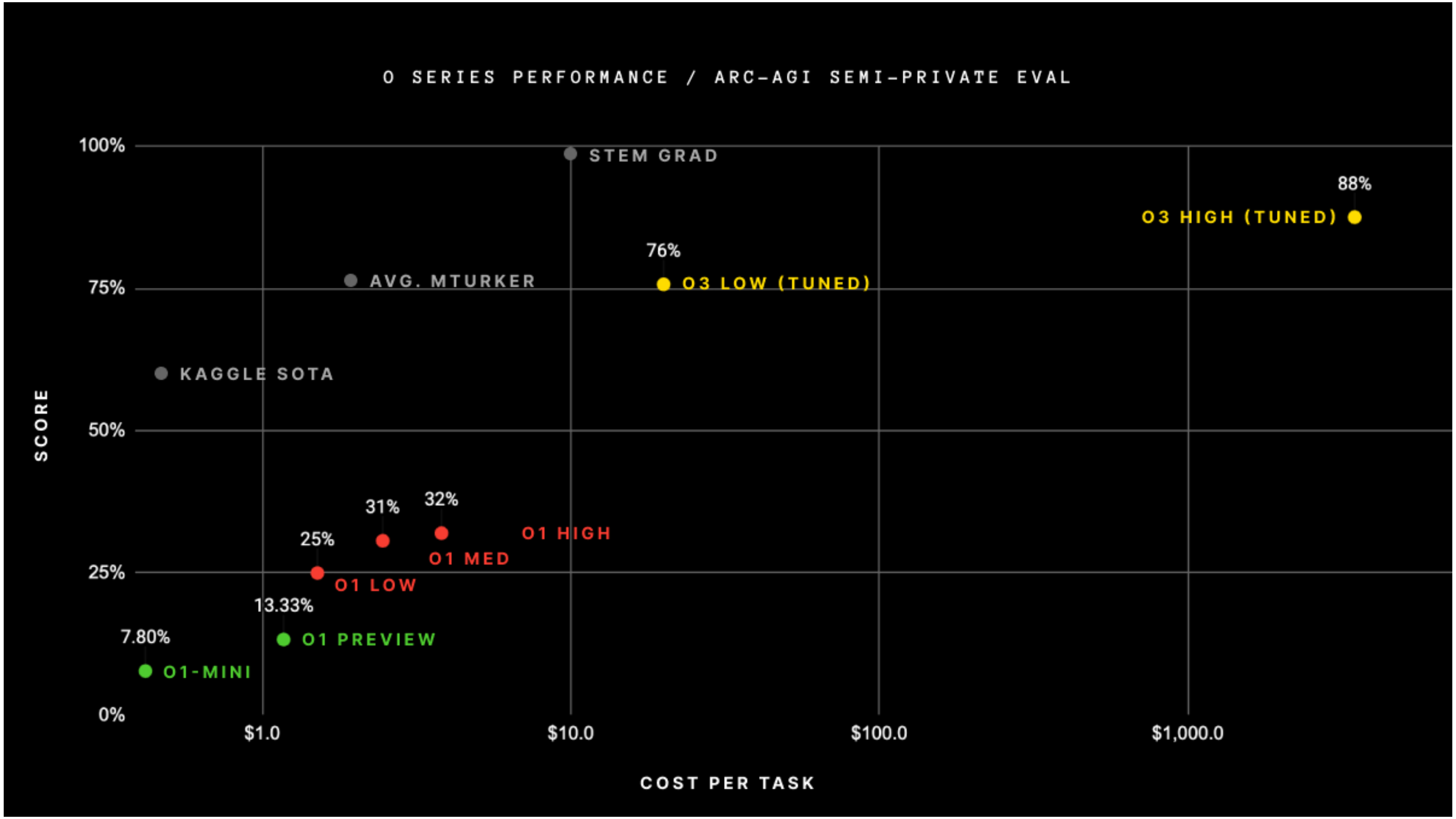
1. Семплируем различные цепочки рассуждений
2. Валидируем цепочки, извлекаем конечный ответ
3. Взаимодействуем с внешним окружением

Learning to reason

OpenAI O1 [точно неизвестно] — имея цепочки рассуждений, давайте на них обучаться…



Inference-Time Scaling



DeepSeek-R1

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think> </think> and <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>. User: prompt. Assistant:

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>
To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$
$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$
Rearrange to isolate the inner square root term:
$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$
...
Wait, wait. Wait. That's an aha moment I can flag here.
Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$
We started with the equation:
$\sqrt{a - \sqrt{a + x}} = x$
First, let's square both sides:
$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$
Next, I could square both sides again, treating the equation: $\cdots$
...

[9] — DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, DeepSeek AI, 2025

DeepSeek-🥴1

Все еще есть куда расти!

Flowers
@flowersslop

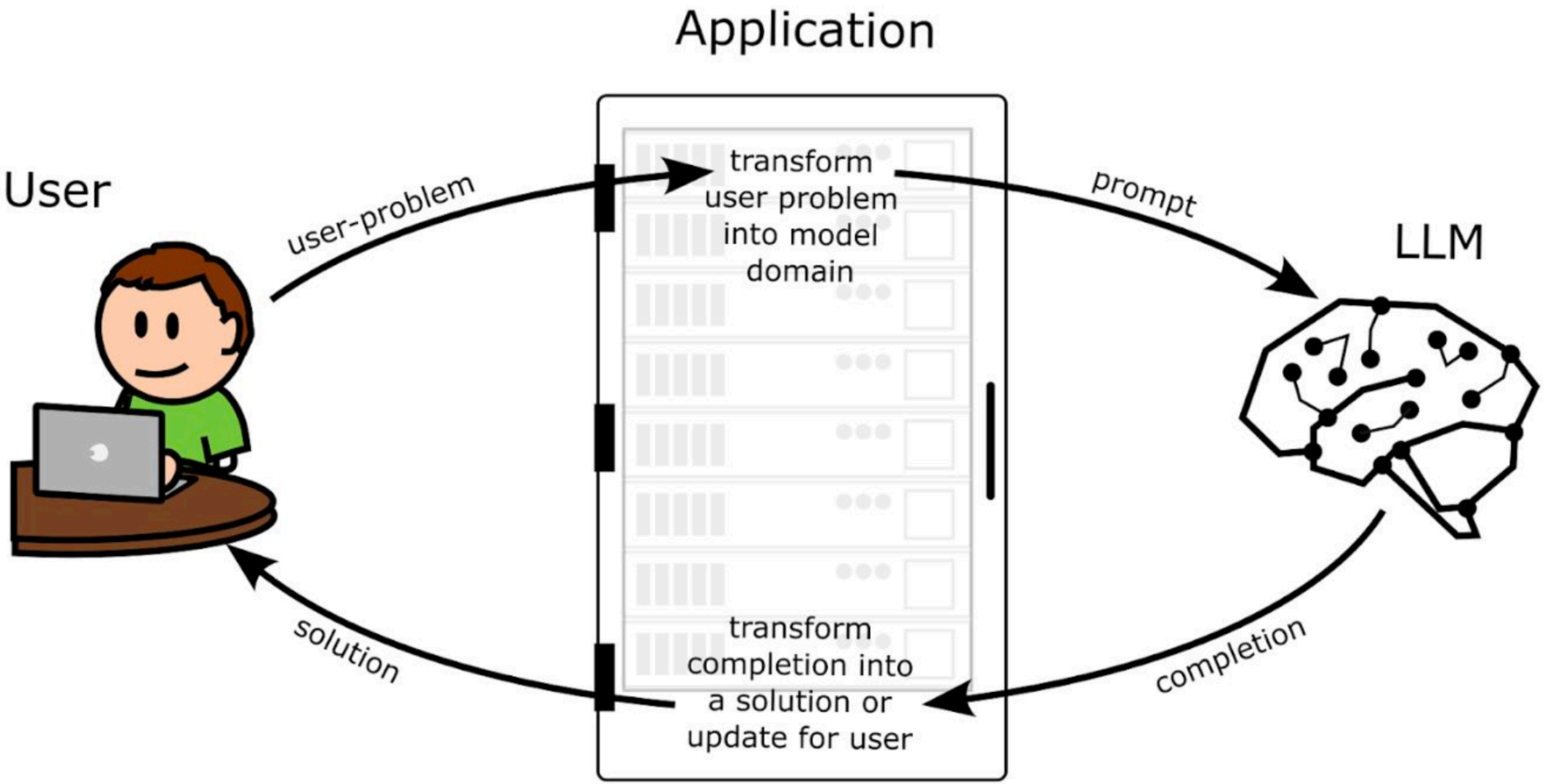
DeepSeek R1 when it is prompted to select a random number:

Okay, the user wants me to think of a number between 1 and 10,000 but not tell them. Hmm, they mentioned not to overthink it, so maybe just pick a random number quickly. Let me see. Well, people often have favorite numbers or patterns. Maybe go with something like 7777? But that's four 7s, which might be too obvious. Alternatively, 1234? Too sequential. Maybe a number that's a common choice but not too predictable. Wait, I should just pick the first number that comes to mind. Let me try that. Closing my eyes and... 42? Wait, that's from Hitchhiker's Guide, but maybe they don't want a reference. Alternatively, 1004? That's within the range. Or 8675? No, that's from a song. Hmm, maybe 3141? Like the first four digits of pi. But 3141 is 3.141... multiplied by 1000. That's clever, but maybe too nerdy. Alternatively, 1984? But that's a book title. Maybe 2023, the current year? But they might not want a recent year. Or 1337, which is leet speak. But that's only 4 digits. Wait, 1337 is 1-3-3-7, which is 1337. Maybe that's too slangy. Alternatively, 5000, the midpoint. But that's too obvious. How about 777? No, that's three digits. Wait, the number needs to be between 1 and 10,000, so up to four digits. Let me just pick 42 again. Wait, that's too low. Maybe 420? But that's associated with cannabis culture. Not sure. Maybe 6969? That's a playful number but might be seen as immature. Alternatively, 1111? A repeating digit. Or 2222? Hmm. Maybe 1234 is too simple. Wait, the user said not to overthink, so just pick a number without second-guessing. Let me go with 7321. No reason, just random digits. But maybe that's too arbitrary. Alternatively, 500, but 0500 is 500. Wait, leading zeros don't count. So 500 is 500. Maybe 8888? That's considered lucky in some cultures. But again, maybe too obvious. Alright, I'm overcomplicating it. The user said not to overthink. Let me just pick the first number that comes to mind. Okay, 7777. Done.

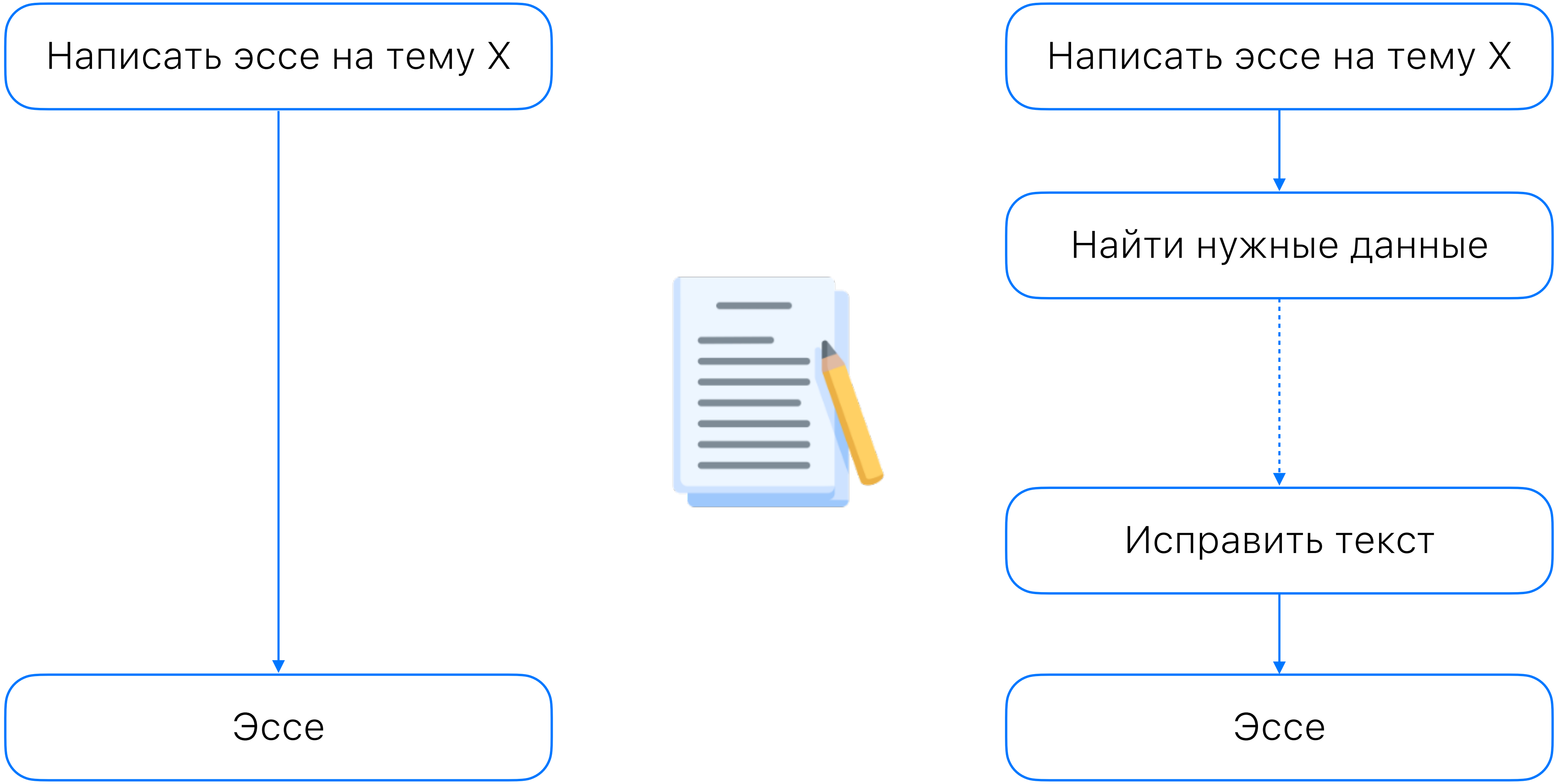
8:40 PM · Jan 22, 2025 · **268.5K** Views

Используем внешний мир 🌎

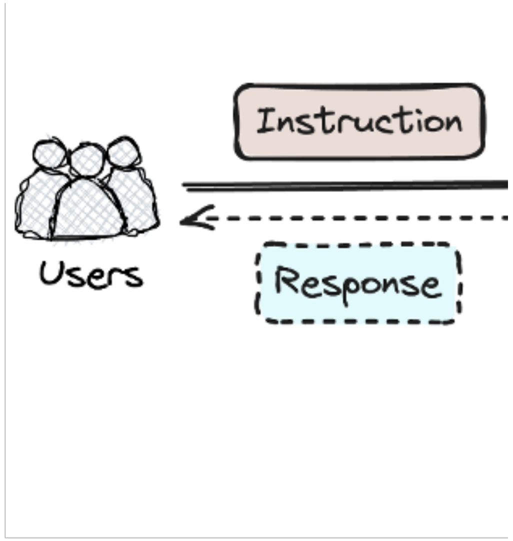
LLM Application



Agentic workflow



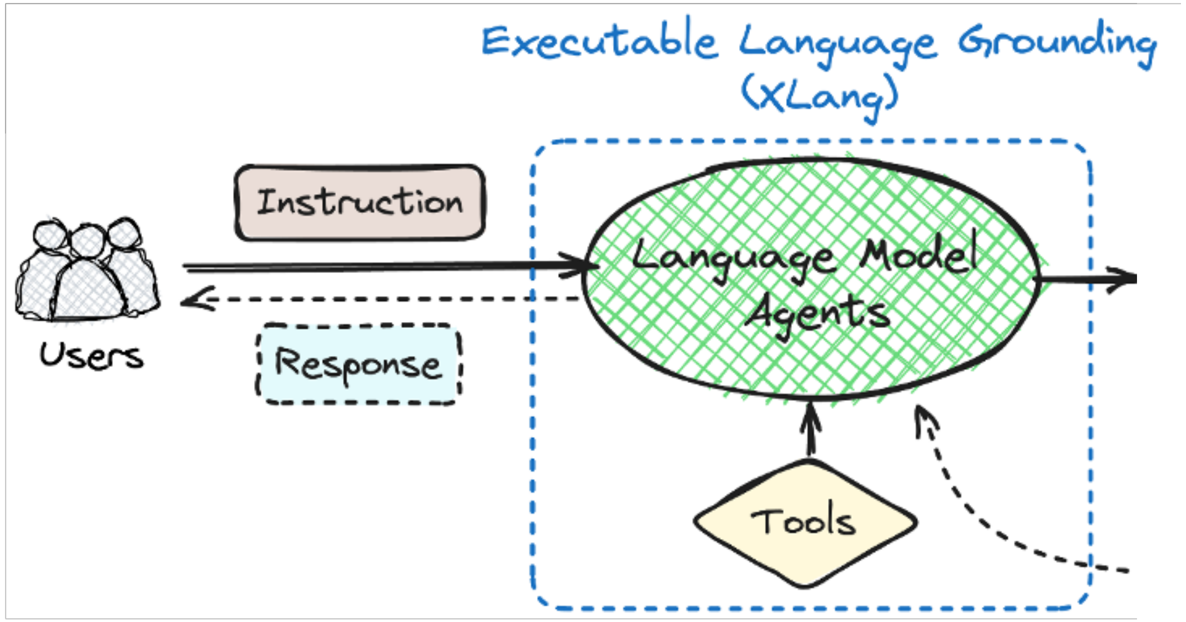
Добавим к LLM инструменты



- 👉 Исполнение кода
- 👉 Википедия
- 👉 Календарь
- 👉 Wolfram Alpha
- 👉 Поисковик
- 👉 Другая LLM
- 👉 CI/CD
- 👉 Почта
- 👉 ...

https://github.com/xlang-ai/xlang-paper-reading

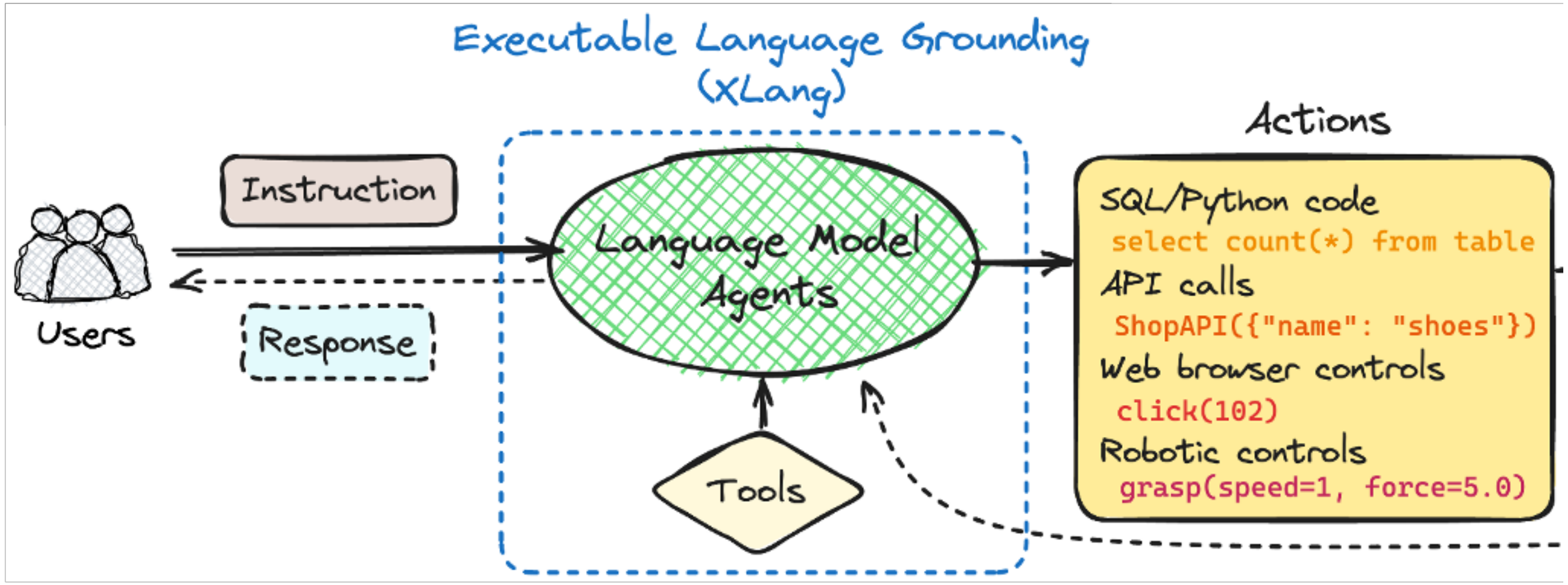
Добавим к LLM инструменты



- 👉 Исполнение кода
- 👉 Wolfram Alpha
- 👉 CI/CD
- 👉 Википедия
- 👉 Поисковик
- 👉 Почта
- 👉 Календарь
- 👉 Другая LLM
- 👉 ...

https://github.com/xlang-ai/xlang-paper-reading

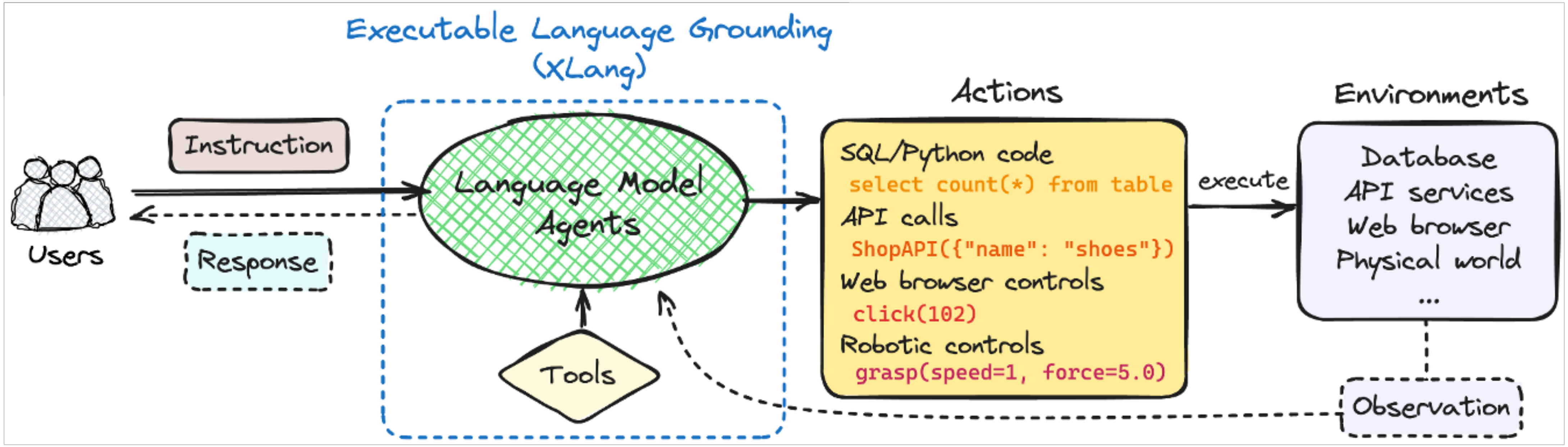
Добавим к LLM инструменты



- 👉 Исполнение кода
- 👉 Wolfram Alpha
- 👉 CI/CD
- 👉 Википедия
- 👉 Поисковик
- 👉 Почта
- 👉 Календарь
- 👉 Другая LLM
- 👉 ...

https://github.com/xlang-ai/xlang-paper-reading

Добавим к LLM инструменты



- 👉 Исполнение кода
- 👉 Wolfram Alpha
- 👉 CI/CD
- 👉 Википедия
- 👉 Поисковик
- 👉 Почта
- 👉 Календарь
- 👉 Другая LLM
- 👉 ...

https://github.com/xlang-ai/xlang-paper-reading

In a nutshell 🌰 | 1. Создание промпта

You have a python function which you can use as a tool:

def create_blank_file(directory, filename):

- """Create blank file
- :param str directory: path to root directory
- :param str filename: name of the blank file
- """

Use this function if necessary as a python function with proper arguments for solving user query. If the user request does not require the function usage return "pass".

In a nutshell 🌰 | 1. Создание промпта

You have a python function which you can use as a tool:

def create_blank_file(directory, filename):
- """Create blank file
- :param str directory: path to root directory
- :param str filename: name of the blank file
- """

Use this function if necessary as a python function with proper arguments for solving user query. If the user request does not require the function usage return "pass".

User query: Please create two files in a folder my_directory with names first.txt and second.txt

In a nutshell 🌰 | 2. Получаем ответ от LLM

You have a python function which you can use as a tool:

def create_blank_file(directory, filename):

- """Create blank file
- :param str directory: path to root directory
- :param str filename: name of the blank file
- """

Use this function if necessary as a python function with proper arguments for solving user query. If the user request does not require the function usage return "pass".

User query: Please create two files in a folder my_directory with names first.txt and second.txt

create_blank_file(my_directory, first.txt)
create_blank_file(my_directory, second.txt)

In a nutshell 🌰 | 3. Запускаем инструмент

You have a python function which you can use as a tool:

def create_blank_file(directory, filename):

- """Create blank file
- :param str directory: path to root directory
- :param str filename: name of the blank file
- """

Use this function if necessary as a python function with proper arguments for solving user query. If the user request does not require the function usage return "pass".

User query: Please create two files in a folder my_directory with names first.txt and second.txt

create_blank_file(my_directory, first.txt)
create_blank_file(my_directory, second.txt)

```
code_snippet="""
create_blank_file(my_directory, "first.txt")
create_blank_file(my_directory, "second.txt")
"""
exec(code_snippet)
```

In a nutshell 🌰 | 3. Запускаем инструмент

```
code_snippet = """create_blank_file(my_directory, first.txt)
create_blank_file(my_directory, second.txt)"""

exec(code_snippet)
```

Function Calling

API часто дает возможность указать доступные инструменты — функции

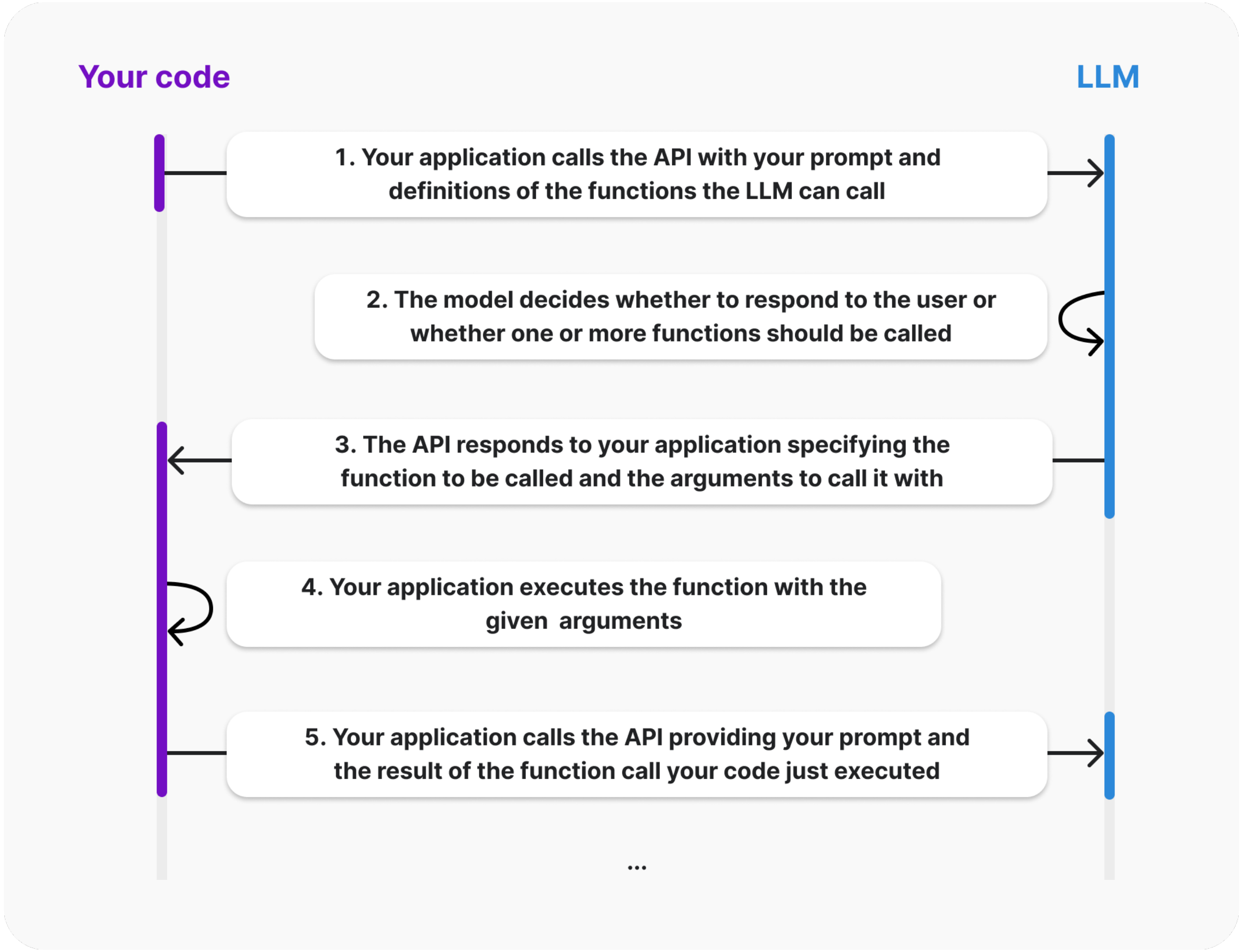
Позволяет получать более достоверные и структурированные ответы от LLM:

```
[{
  "id": "call_12345xyz", "type": "function",
  "function": {
    "name": "get_weather",
    "arguments": "{'location':'Paris'}"
  }
}]
```

https://platform.openai.com/docs/guides/function-calling

```python
from openai import OpenAI

client = OpenAI()

tools = [
  {
      "type": "function",
      "function": {
          "name": "get_weather",
          "parameters": {
              "type": "object",
              "properties": {
                  "location": {"type": "string"}
              },
          },
      },
  }
]

completion = client.chat.completions.create(
  model="gpt-4o",
  messages=[{
    "role": "user",
    "content": "What's the weather like in Paris today?"
  }],
  tools=tools,
)

print(completion.choices[0].message.tool_calls)
```

Function Calling Pipeline



Дополнительно:

- 👉 Указывать required поля
- 👉 Возвращать результат выполнения функции
- 👉 Parallel Function Calling
- 👉 ...

Изучаем документацию

🦜🔗 LangChain

Фреймворк для разработки приложений на основе LLM

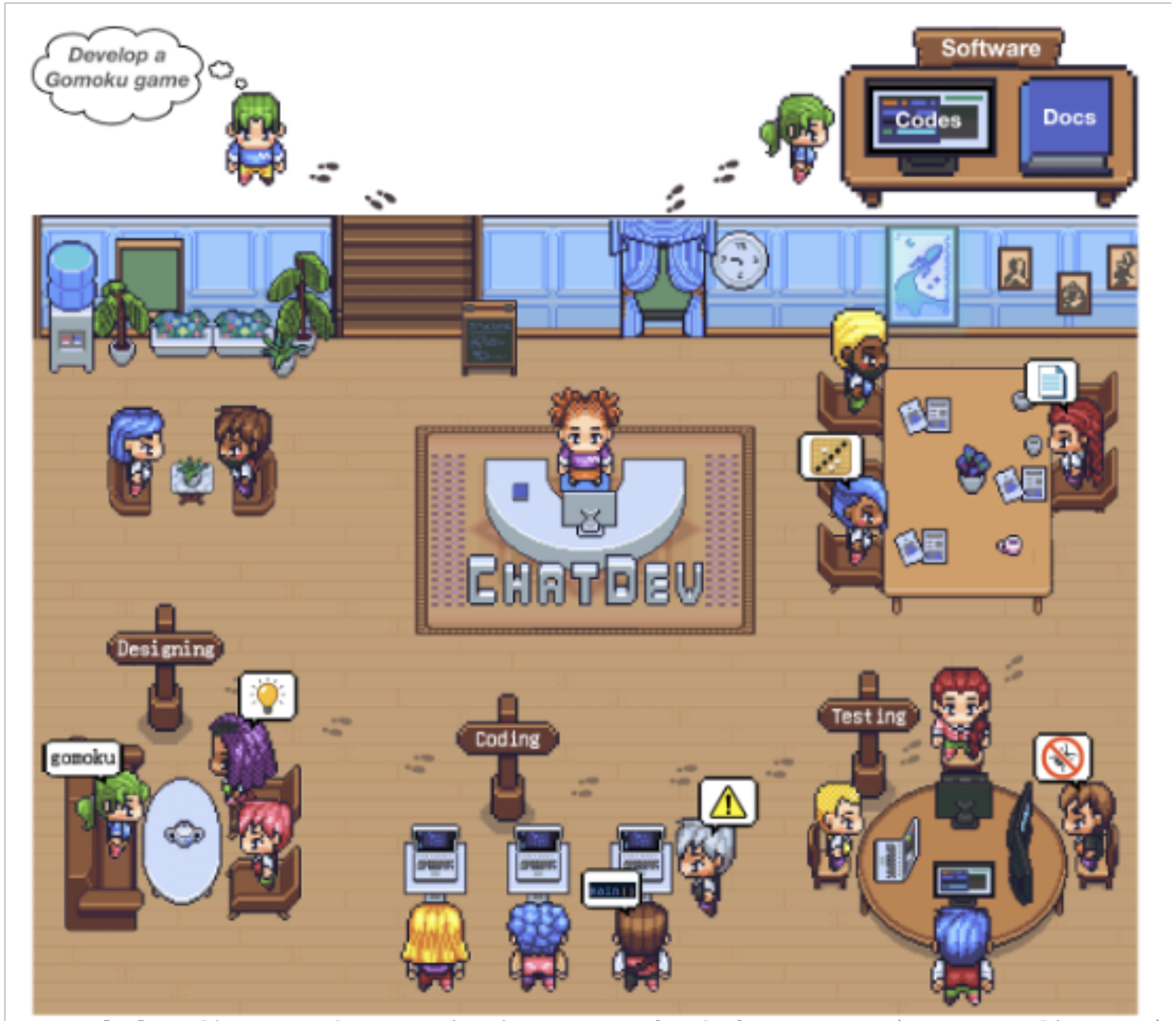
- 👉 Интеграция со множеством API и LLM провайдерами: OpenAI, Anthropic, HF, ...
- 👉 Компоненты для работы с IO, Retrieval и агентами, промптами
- 👉 LangGraph — stateful приложения

[GitHub: langchain]

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

tagging_prompt = ChatPromptTemplate.from_template(
"""Extract the desired information from the following passage.
Only extract the properties mentioned in the 'Classification' function.

Passage:
{input}
""")

class Classification(BaseModel):
    sentiment: str = Field(description="The sentiment of the text")
    language: str = Field(description="The language the text is written in")
    aggressiveness: int =
     Field(description="How aggressive the text is on a scale from 1 to
     10")

llm = ChatOpenAI(temperature=0, model="gpt-4o-mini")\
        .with_structured_output(Classification)



>>> inp = "Estoy increiblemente contento de haberte conocido! Creo que
seremos muy buenos amigos!"
>>> prompt = tagging_prompt.invoke({"input": inp})
>>> llm.invoke(prompt)
Classification(sentiment='positive', aggressiveness=1, language='Spanish')
```

Давайте жить дружно ❤️

Счастливый мир будущего

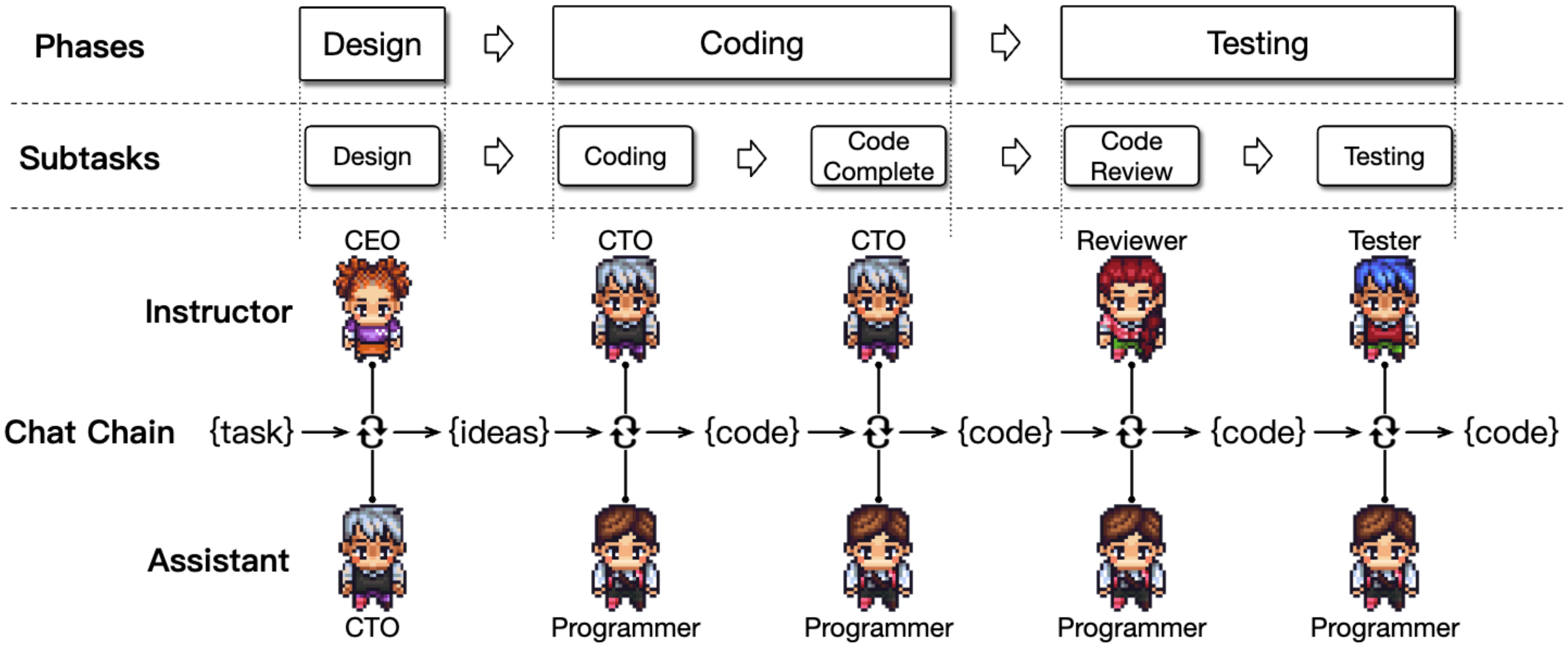


LLM-based application на основе множества отдельных агентов:

1. Декомпозируем задачу на отдельные части
2. Для каждой задачи определим свою отдельную LLM
3. Заставим агентов взаимодействовать друг с другом для решения крупной задачи

[11] — ChatDev: Communicative Agents for Software Development, Qian et al., Tsinghua University, 2024

Можем разработать целое приложение с LLM*



* — если повезет

Ресурсы

Учение — свет, неучение — тьма.

Что не успели разобрать:

👉 Галлюцинации — какие бывают и как бороться

👉 Атаки на LLM и защиты от инъекций

👉 Интеграция агентов в готовые инструменты

Полезные ресурсы:

1. Large Language Model Agents (MOOC)
2. Advanced LLM Agents (MOOC)
3. Prompt Engineering Guide

На этом все🤝

Егор Спирин — vk.com/boss
VK Lab — vk.com/lab